

Agent Harness

从执行脚手架到自我进化系统

内部研究稿

资料截面：2026-08-22

序：为什么现在需要一本 Harness 小书

同一个模型放进不同 Agent 产品，表现可能像两个不同的系统。差异来自模型之外的上下文、工具、环境、权限、反馈、恢复和验证。本书把这组能力统称为 Agent Harness。

Harness 不是一个框架品牌，也不只是 `while model calls tools`。它是模型与真实世界之间的运行时和治理层：把人的意图编译成任务，把环境状态编译成上下文，把模型动作变成受控副作用，再把外部结果编译成证据。

本书面向企业 Agent 平台架构师和高级工程师。历史篇解释从规划、BDI、ReAct 到 coding agent 的转折；原理篇给出状态机、上下文、工具、安全、验证和多 Agent 设计；产品篇研究 Claude Code、Codex、Cursor、DeepSeek Harness 与 OpenHands；进化篇区分任务内、跨任务、Harness 和模型四层；实践篇给出供应商无关参考架构和迁移路线。

全书的核心公式是：

$$\text{Agent System Capability} = \text{Model} \times \text{Harness} \times \text{Environment} \times \text{Feedback}$$

乘法意味着任一项接近零，整体都可能失败。强模型无法弥补被截断的关键上下文、危险工具、损坏环境或错误 verifier；厚 Harness 也不能无限跨越模型能力边界。

产品事实按 2026-08-22 的公开材料记录。快速变化的功能会过时，设计原则应更稳定。书中区分公开事实、论文结果和作者综合判断；内部使用不改变证据要求。

目录

自动目录

序：为什么现在需要一本 Harness 小书

目录

第一篇 历史：Agent 如何从会回答变成会行动

第一章 从控制循环到 Agent Runtime

1. 规划：把目标翻译成动作序列
2. 反应与承诺：BDI 的长期遗产
3. 多 Agent：1980 年已经出现的协调税
4. 从手写控制器到模型控制器
5. ReAct：现代最小循环的形成
6. Toolformer、Reflexion 与 Voyager：三种能力迁移
7. Harness 的历史不是功能累积，而是责任迁移

第二章 2023：自主 Agent 爆发与第一次祛魅

1. BabyAGI：任务队列就是最小外部认知
2. AutoGPT：把开放动作空间交给语言模型
3. LangChain：把 Agent 拆成可复用抽象
4. AutoGen：把 workflow 表达成多 Agent 对话
5. 第一次祛魅：为什么 Demo 自治不能直接进入生产
 - 5.1 语言流畅度与状态正确性错配
 - 5.2 语义相似与记忆有效性错配
 - 5.3 工具可调用与动作可授权错配
 - 5.4 循环持续与目标进展错配
 - 5.5 自我批评与独立验证错配
 - 5.6 多角色对话与多样性错配
6. 从 Framework 到 Runtime
7. 这一代系统留下了什么

第三章 接口也是智能：Aider、SWE-agent 与 OpenHands

1. Aider：上下文不是越多越好
2. 编辑格式：工具接口会占用模型能力
3. SWE-agent：Agent-Computer Interface 成为设计对象
4. CodeAct：代码作为动作语言
5. OpenHands：把 Agent 与 Runtime 分开
6. Event Stream 不等于完整事件溯源

7. Benchmark 开始评价 “模型 × Harness”

8. 从接口工程得到的七条原则

第四章 Coding Agent 转折：真实仓库成为反馈环境

1. 从聊天记录到工作树

2. 可执行反馈改变了规划

3. CodeAct 与统一动作空间

4. OpenHands 的分离

5. Benchmark 的双重作用

6. 现代 Harness 的形成

第二篇 原理：生产级 Harness 的构成

第五章 Agent = Model × Harness × Environment × Feedback

1. Model：概率性策略与生成器

2. Harness：运行时中介与控制系统

3. Environment：不是一个 shell，而是任务世界

4. Feedback：观测不等于评价

5. 为什么使用乘法而不是加法

6. Workflow、Agent 与 Harness 的关系

7. 控制面、数据面与执行面

7.1 控制面

7.2 数据面

7.3 执行面

8. 概率性建议与确定性约束

9. Harness 的厚与薄

10. 一个可实现的最小分层

11. 本书的统一分析模板

第六章 Agent Loop：从 while 循环到持久状态机

1. Thread、Turn、Step 与 Attempt

2. 最小状态机

3. 模型停止不等于任务完成

4. Streaming 是事件协议，不只是打字动画

5. 取消必须贯穿调用链

6. Timeout、Retry、Resume 不是一回事

7. 副作用账本与不确定提交

8. Checkpoint 应保存什么

9. Compaction 是有损状态迁移

10. Error Taxonomy 决定恢复策略

11. 并行工具与一致性

12. 一个更完整的循环伪代码

13. 最小可靠性测试集

第七章 上下文、缓存、压缩与记忆

1. 五种必须分开的信息
2. Context Assembly 是一次编译
3. 长窗口不是无限注意力
4. 静态上下文与动态发现
5. Prompt Cache 是架构约束
6. Compaction 是有损编译，不是删除旧消息
7. MemGPT 与分层记忆
8. 记忆类型与写入权限
9. Memory Poisoning 与程序漂移
10. 检索不是只有向量相似度
11. Skills 不是 Memory 的别名
12. Context Quality 的评价
13. 推荐的上下文架构
14. 最小验收清单

第八章 工具、ACI、MCP 与 Code Mode

1. Tool Definition 只是起点
2. 好工具的十个条件
3. 错误协议是 ACI 的一部分
4. Tool Result 不应只有字符串
5. MCP 解决的是互操作，不是全部 Harness 问题
6. MCP 的安全边界
7. Tool Discovery：工具也需要分页
8. CLI：最通用但最难治理的工具总线
9. Native Tool Call 与 Code Mode
10. 并行工具的调度语义
11. 工具版本与动态变化
12. Tool Policy 与 Tool Execution 分离
13. 如何评价工具层
14. 企业平台的工具分层
15. 最小工具契约伪代码

第九章 权限、沙箱、凭证与供应链

1. 四个不同问题
2. Prompt 不是安全边界
3. 从逐次批准到受控自治
4. 文件系统与网络必须同时限制
5. Policy 决策模型

6. 凭证不进入模型上下文
7. Prompt Injection 的系统应对
8. 多 Agent 的权限传播
9. MCP 与插件供应链
10. 审批 UX 是安全系统
11. 审计记录与模型 trace 分离
12. 数据分类与跨域流动
13. 风险分级
14. 安全测试
15. 安全参考边界

第十章 验证、完成契约与证据包

1. Stop、Answer、Success 与 Commit 是四件事
2. 完成契约从任务入口开始
3. 验证金字塔
 - 3.1 确定性检查
 - 3.2 环境与结果检查
 - 3.3 模型检查
 - 3.4 人类检查
4. Benchmark 也是会腐化的软件
5. 非确定性系统不能只跑一次
6. 防止测试投机与 verifier 篡改
7. 证据包是交付协议
8. 完成门的参考状态机
9. 三类案例
 - 9.1 仓库软件工程
 - 9.2 企业数据分析
 - 9.3 自我进化 Agent
10. 常见失败模式
 - 10.1 把 Agent 自述当证据
 - 10.2 只验证最终文本
 - 10.3 同一主体控制目标、实现与评分
 - 10.4 失败后动态降低门槛
 - 10.5 迷信 benchmark 排名
 - 10.6 无限验证—修复循环
11. 企业落地清单

第十一章 多 Agent、委派与协作拓扑

1. 先区分五种经常混淆的东西
2. 多 Agent 的收益来自哪里

3. 何时不应使用多 Agent
4. 四类基本拓扑
 - 4.1 Manager—Worker
 - 4.2 Pipeline / Assembly Line
 - 4.3 Peer Handoff
 - 4.4 Blackboard / Event Graph
5. 委派是一份受限合同
6. 结果必须是 artifact，不是意见
7. 共享状态与并发写入
8. 错误如何在团队中传播
9. Debate、Critique 与 Ensemble
10. 调度、背压与预算
11. 权限、身份与责任链
12. 可观测性：同时看到树和因果链
13. 三类贯穿案例
 - 13.1 仓库软件工程
 - 13.2 企业数据分析
 - 13.3 自我进化 Agent
14. 一个最小参考实现
15. 设计原则总结

第十二章 可观测性、轨迹与评测运营

1. 轨迹是事件图
2. 指标分四层
3. 失败分类先于优化
4. Eval 是持续运营系统
5. 在线监控与离线评测闭环
6. Trace replay 的边界
7. 运营仪表盘

第三篇 产品：当代主流 Harness 的不同答案

第十三章 Claude Code：薄决策环与厚运行时

1. 环境优先
2. 工具与扩展
3. 权限与沙箱
4. 产品取舍
5. 企业集成方式

第十四章 OpenAI Codex：协议化 Agent 核心与工程控制面

1. App Server 是关键边界
2. 工作树与并行

3. 策略层
4. Harness Engineering
5. 取舍与借鉴

第十五章 Cursor：IDE 原生上下文与云 Agent

1. 动态上下文发现
2. Harness 与模型共同适配
3. 云 Agent 的环境工程
4. IDE 人机协作的优势与风险
5. 架构启示

第十六章 DeepSeek Harness：可组合运行时与进化载体

1. Cordis 插件树
2. 分层配置与 scope
3. Tool Runtime 与 Code Mode
4. 可变不等于会进化
5. 企业取舍

第十七章 OpenHands：Agent 与 Runtime 分离的开放架构

1. Action—Observation—Event
2. Runtime 生命周期
3. 开放平台的价值
4. 与其他案例的互补
5. 企业采用方式

第十八章 横向比较：不同 Harness 的答案

1. 薄与厚不是优劣
2. 本地与云
3. 开放与闭源
4. 共同收敛
5. 选择原则

第四篇 进化：Agent 如何从轨迹中变得更好

第十九章 进化不是自我修改：目标函数、数据与治理

1. 四个层次
2. 优化对象与根信任
3. 数据不是天然经验
4. 多目标而非单分数
5. 最小可信闭环
6. 能力边界

第二十章 任务内进化：搜索、反思与验证—修复

1. Reflexion 的贡献与边界
2. 搜索不是无限重试

3. 验证—修复循环
4. 动态工作流
5. 三类案例

第二十一章 跨任务经验化：Memory、Skill 与策略库

1. 四类持久对象
2. 写入门比检索更重要
3. 检索与适用条件
4. Skill 供应链
5. 遗忘与纠错

第二十二章 Harness 进化：Prompt、工具、上下文与工作流

1. 先归因再变异
2. 最小变更原则
3. 质量多样性
4. 四组门禁
5. 发布

第二十三章 模型进化：轨迹蒸馏、偏好与强化学习

1. 轨迹不等于训练样本
2. SFT 与蒸馏
3. 偏好优化
4. RL 与可验证奖励
5. 模型—Harness 2×2 归因

第二十四章 受控进化闭环：门禁、灰度、回滚与反投机

1. 双平面架构
2. 实验协议
3. 防 Reward Hacking
4. Canary 与回滚
5. 组织责任
6. 进化账本

第五篇 实践：下一代企业 Harness

第二十五章 三个贯穿案例的端到端设计

1. 仓库级软件工程
2. 企业数据分析
3. 自我进化 Agent
4. 共用骨架

第二十六章 下一代企业 Harness 参考架构

1. 六层结构
2. Canonical contracts
3. 身份与租户

- 4. Durable execution
- 5. Policy 与 sandbox
- 6. Evidence-first completion
- 7. 演进边界
- 8. 非功能要求

第二十七章 Agent SDD：规范驱动的任务与发布

- 1. 规范层级
- 2. 从意图到合同
- 3. Specification as environment
- 4. 发布门
- 5. 规范债务

第二十八章 成熟度模型与 Build-vs-Buy

- 1. 五级成熟度
- 2. 买什么
- 3. 自建什么
- 4. 何时自研 runtime
- 5. 决策矩阵

第二十九章 从接入现有 Agent 到自研运行时

- 阶段一：封装而非散接
- 阶段二：外置完成与证据
- 阶段三：统一执行面
- 阶段四：建立评测基线
- 阶段五：逐层替换
- 阶段六：引入受控进化
- 迁移反模式

第三十章 展望：Harness OS、Agent 组织与持续进化

- 1. 模型与 Harness 共同设计
- 2. 从应用到 Agent 组织
- 3. 环境成为主要护城河
- 4. 持续进化的现实形态
- 5. 新风险
- 6. 最终判断

附录

附录 A：语言无关核心接口

附录 B：Harness 架构评审检查表

- 任务与完成
- 上下文与记忆
- 工具与环境

权限与安全

Durable 与多 Agent

Eval 与进化

附录 C：术语表

研究方法与局限

完整参考文献

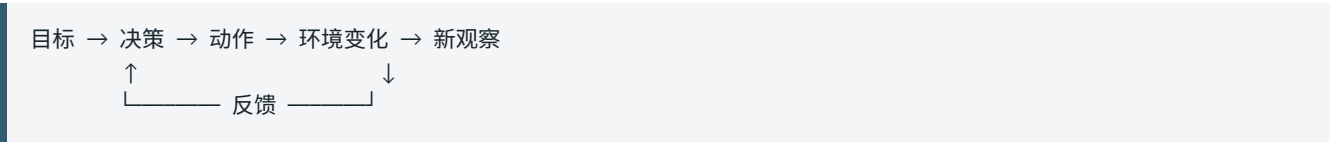
Markdown 章节按下列五篇排列。

第一篇 历史：Agent 如何从会回答变成会行动

第一章 从控制循环到 Agent Runtime

本章状态：正文初稿 v0.1；事实和引用将在全书审计阶段再次核验。

如果把 Claude Code、Codex 或 DeepSeek Harness 的界面全部拿掉，剩下来的核心似乎简单得令人失望：接收目标，调用模型，执行模型选择的动作，把结果送回模型，如此循环，直到完成或耗尽预算。



这个循环并不新。控制论研究反馈，自动规划研究如何从初始状态抵达目标状态，机器人研究感知与行动，BDI 架构研究信念、目标与承诺，多智能体系统研究任务分配与通信协议。现代 Harness 的新意不在于重新发明循环，而在于把一种高能力、概率性、上下文受限、可能调用任意软件工具的语言模型，放进真实计算环境后，为它补上可靠运行所需的工程结构。

因此，要理解 Harness，最好的起点不是 2023 年的 AutoGPT，而是三个更早的问题：机器如何形成行动序列？一个持续运行的 Agent 如何在变化环境中保持承诺并修正行为？多个自治执行者如何分工而不依赖全局共享状态？

1. 规划：把目标翻译成动作序列

1971 年的 STRIPS 将规划描述为：在一个世界模型中寻找一串操作，使初始状态经过这些操作后满足目标公式。它把动作表示成具有前置条件和状态效果的操作符，使“怎样完成任务”从一段专用程序转化成可以搜索的状态转换问题。[STRIPS 原始论文](#)

用现代 Harness 的语言重写，STRIPS 已经包含四个熟悉成分：

State	当前环境的结构化描述
Goal	希望满足的终止条件
Action	带前置条件和效果的环境操作
Planner	搜索可行 Action 序列的控制器

现代 coding agent 的文件读取、补丁、命令执行和测试工具，也可以被视为动作。不过，两者之间存在决定性差异。经典规划通常假设动作语义明确、状态足够可知、执行结果近似确定；软件 Agent 面对的仓库、依赖、网络服务和人类意图却是不完全可观测的。模型甚至可能误解工具说明、生成无效参数，或者把命令成功退出误认为业务目标已经完成。

这解释了为什么现代 Agent 很少只生成一份完整计划再机械执行。更常见的是滚动规划：先选择一个信息增益高或风险可控的动作，观察真实结果，再更新局部计划。ReAct 后来把这种交替模式显式写进语言模型轨迹，但其深层问题仍是规划与控制中的老问题：当世界模型不完整时，计划必须服从反馈。

对 Harness 设计者而言，这段历史留下第一条原则：

计划不是事实，而是一个必须持续接受环境证据修正的假设。

这意味着 plan mode 可以帮助形成意图和审查范围，却不应成为独立于执行反馈的僵硬工作流。真正可靠的 Harness 必须保留重新观察、修订计划、撤销局部动作和报告不可达目标的通道。

2. 反应与承诺：BDI 的长期遗产

只会在每一步对刺激作出反应的系统容易漂移；只会遵循预先计划的系统又难以适应变化。Belief-Desire-Intention (BDI) 架构试图在两者之间建立实际推理循环：Agent 根据对世界的信念识别可能目标，从中形成当前愿望或目标，再选择并承诺某些意图；新事件到来后，它更新信念、判断现有意图是否仍可行，并决定继续、重规划或放弃。

BDI 的价值不在于要求现代 Harness 必须建立名为 `beliefs`、`desires`、`intentions` 的三个对象，而在于它揭示了三类经常被聊天记录混为一谈的状态：

- 观察和推断出来的世界状态；
- 用户希望实现但尚未承诺执行路径的目标集合；
- 当前已经投入资源、应跨多个步骤保持一致的执行承诺。

早期 PRS、dMARS 与 AgentSpeak 等系统还发展出计划库、事件触发、元级控制和显式 `deliberation cycle`。BDI 综述将其核心贡献概括为：在动态、不可预测环境中平衡主动目标与被动响应，并通过意图表达对未来行为的承诺。[BDI 架构综述](#)

这与现代 Agent 的几个常见故障直接相关。

第一，只有对话历史、没有显式任务状态时，模型可能在长会话中忘记用户真正批准了什么。第二，计划、待办、执行事实和模型猜测如果使用同一种自然语言表示，压缩后很容易相互污染。第三，出现新消息时，如果 Harness 不区分“补充信息”“优先级变化”“取消请求”和“新任务”，Agent 就可能错误地放弃或延续已有承诺。

因此，企业 Harness 至少应在运行时区分：

<code>ObservedState</code>	可追溯到环境或用户消息的事实
<code>Hypothesis</code>	模型尚未验证的解释
<code>Goal</code>	用户或上层系统定义的结果条件
<code>Commitment</code>	已批准并正在执行的目标/约束
<code>Plan</code>	当前可替换的动作建议
<code>ExecutionState</code>	已开始、等待、取消、失败或完成

这里最重要的不是命名，而是不同状态拥有不同的更新权限和证据要求。模型可以自由提出假设和计划，却不应自行把假设升级为事实，也不应把未批准目标升级为承诺。这正是现代 Harness 中“概率性认知”和“确定性控制”分层的早期思想来源。

3. 多 Agent：1980 年已经出现的协调税

1980 年的 Contract Net Protocol 研究松耦合节点如何通过协商分配任务：拥有任务的 manager 发布任务描述，潜在 contractor 根据自身能力和资源提出方案，manager 再授予合同。该系统强调没有全局共享数据和单一全局控制，任务分配必须同时考虑资源利用与问题求解焦点。[Contract Net 原始论文](#)

这与今天的 subagent、agent team 和异步云 Agent 非常相似，但它也提醒我们，多 Agent 从来不是免费的并行计算。一次委派至少需要：

1. 切分出边界足够清楚的任务；
2. 描述输入、输出、权限和完成条件；
3. 选择具备适当能力与资源的执行者；
4. 传递必要上下文，同时避免复制整个主会话；
5. 接收结果并验证，而不是把子 Agent 的自然语言结论当作事实；
6. 处理超时、重复执行、局部失败和相互冲突的修改；
7. 把结果重新合并到主任务状态。

如果拆分收益小于这些协调成本，多 Agent 会比单 Agent 更慢、更贵、更难调试。这也是为什么“模型能够调用 subagent”不等于系统具备良好的多 Agent 架构。真正的设计对象是委派协议、隔离边界、结果契约、取消传播和合并策略。

现代 coding agent 使用 worktree、独立容器或远程 workspace，不只是为了方便并行，而是在物理上减少共享可变状态。这个选择与 Contract Net 的松耦合假设一脉相承：信息通过协议传递，执行者不依赖一个可任意读写的工作台。

4. 从手写控制器到模型控制器

传统 Agent 的策略、计划选择和异常处理通常由规则、搜索算法或专用程序实现。大语言模型改变的是控制器的表达能力：它可以读取自然语言目标和非结构化观察，在没有为每种任务编写专用规则的情况下，提出下一动作并生成工具参数。

这带来显著的泛化能力，也引入四种结构性不确定性：

- **语义不确定性**：模型可能误解目标、上下文或工具描述；
- **动作不确定性**：它可能选择错误工具或构造无效参数；
- **状态不确定性**：上下文窗口只包含环境的一部分，并可能在压缩中失真；
- **完成不确定性**：模型说“完成”只是一项预测，不是外部世界已经满足目标的证明。

所以，现代 Harness 不能只是把模型接在 shell 上。它需要用确定性软件包围概率性决策：工具 schema 约束动作形状，权限策略约束可执行范围，沙箱约束副作用，日志保存轨迹，预算限制循环，测试和评估器判断结果，人工审批承接不可自动化的价值判断。

可以把两者的分工写成一条简单边界：

模型负责：提出、解释、比较、生成、诊断
Harness负责：授权、执行、隔离、记录、计量、验证、恢复

这不是说 Harness 中不能存在模型评审器，也不是说控制逻辑必须完全固定。关键是：任何影响安全、资源、归因和进化晋级的决定，都不能只依赖被评对象的一次自我陈述。

5. ReAct：现代最小循环的形成

ReAct 将 reasoning trace 与 action 交替生成：推理帮助模型维护和调整计划，动作让模型从外部知识源或环境取得新信息，新观察再进入后续推理。[ReAct](#)

它为现代工具调用 Agent 提供了极具影响力的最小模板：

```
while budget_available:
    decision = model(context, available_tools)

    if decision.is_final:
        return decision.answer

    validated_call = validate(decision.tool_call)
    observation = execute(validated_call)
    context.append(observation)
```

这个伪代码能运行，却还不是企业 Harness。它没有回答：上下文从哪里来，过长时如何压缩；工具结果是否可信；命令在哪台机器、以谁的身份执行；权限是静态的还是可临时扩展；进程崩溃后如何恢复；如何取消正在执行的工具；怎样判断模型陷入循环；最终答案需要哪些外部证据；多个用户和 Agent 如何隔离；历史轨迹如何进入评测和进化。

换句话说，ReAct 给出了发动机循环，却没有给出整辆车的制动、仪表、车身、道路规则和维修体系。Harness 的历史，就是这些“循环之外的结构”逐渐变成第一等工程对象的历史。

6. Toolformer、Reflexion 与 Voyager：三种能力迁移

2023 年的几项工作分别展示了 Agent 能力可以被迁移到不同位置。

Toolformer 研究如何通过自监督数据让模型学习何时调用 API、调用哪个 API、传什么参数以及如何吸收工具结果。[Toolformer](#) 它把一部分工具选择能力迁移进模型权重。

Reflexion 不更新权重，而是把任务反馈转成语言反思，保存到情景记忆中，影响后续尝试。[Reflexion](#) 它把一部分学习迁移到跨回合上下文。

Voyager 则把成功行为保存为可执行技能库，并结合自动课程、环境错误和自验证持续扩充技能。[Voyager](#) 它把一部分能力迁移到外部、可组合、可复用的程序资产。

这三种路线构成理解“Agent 如何进化”的早期坐标系：

能力沉淀位置	典型机制	优点	主要风险
模型权重	训练、微调、强化学习	推理时直接、可泛化	成本高、难回滚、归因困难
运行时记忆	反思、经验摘要、状态图	更新快、无需改权重	污染、过期、检索误配
外部技能	代码、工具、工作流、插件	可审查、可组合、可版本化	供应链风险、接口漂移
当前任务轨迹	retry、search、verify-fix	即时纠错	成本膨胀、循环与自证偏差

本书后面的“进化篇”将沿着这四层展开。这里先保留一个重要结论：自我改进不必等同于修改模型，也不必等同于 Agent 任意重写自己的运行时。最有工程价值的进化，往往是把一次成功中可验证、可迁移的部分，沉淀到风险最低且最容易回滚的层级。

7. Harness 的历史不是功能累积，而是责任迁移

回看这条历史线，可以发现 Harness 不是一张越来越长的功能清单，而是一系列责任在模型、控制器、环境与人之间重新分配的过程：

经典规划：控制器显式搜索动作序列
BDI： 控制器管理信念、目标与承诺
多 Agent： 协议管理委派与资源协商
ReAct： 模型参与滚动决策与工具选择
Reflexion： 经验进入外部记忆
Voyager： 成功行为进入可复用技能
现代 Harness： 确定性运行时治理概率性控制器

今天看似全新的问题——上下文工程、subagent、skills、tool schema、self-evolution——都能在早期 Agent 研究中找到结构相似物。但结构相似不等于工程问题已经解决。语言模型让动作空间、任务空间和接口空间同时扩大，使过去可以写死在专用系统里的约束，必须升级为通用运行时机制。

这就是 Harness 成为独立层的原因。模型越通用，环境越开放，循环越长，Harness 承担的责任反而越厚：它必须让系统知道自己看见了什么、承诺了什么、能做什么、做过什么、是否真的完成，以及从结果中允许学到什么。

下一章将进入 2023 年的“自主 Agent 爆发期”：AutoGPT、BabyAGI、LangChain 和 AutoGen 如何把循环、记忆、计划与多 Agent 编排带入大众开发实践，又为何很快暴露出失控循环、抽象泄漏、观测不足和生产基础设施缺失的问题。

第二章 2023：自主 Agent 爆发与第一次祛魅

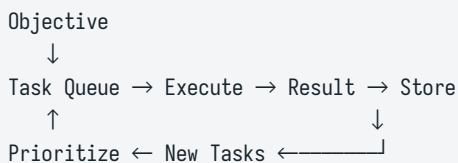
本章状态：正文初稿 v0.1。

2023 年春天，GPT-4 与廉价 API、开源代码和社交媒体演示共同触发了一次“自主 Agent”爆发。AutoGPT、BabyAGI、AgentGPT 等项目让普通开发者第一次直观看见：只要给模型一个目标、少量工具、一段循环和某种记忆，它似乎就能自行拆解任务、搜索网络、写文件、运行代码，并不断决定下一步。

从今天回看，这批系统并没有建立可靠的通用自治。但称它们只是“玩具”同样不准确。它们完成了一次重要的公共实验：把语言模型从单次问答移入持续执行循环，并在极短时间内暴露出 Harness 工程真正困难的部分。

1. BabyAGI：任务队列就是最小外部认知

原始 BabyAGI 的结构极其简洁：从队列取出一个任务，调用执行 Agent，将结果写入记忆，根据目标和最新结果创建新任务，再重新排序任务队列，如此循环。[原始归档代码](#)



它的重要启示不是“需要三个角色提示词”，而是模型之外必须存在可检查的任务状态。如果所有计划只存在于对话文本中，系统很难回答：还有哪些任务？哪个任务正在执行？为什么优先做它？某个结果由哪次执行产生？

任务队列把一部分认知外置成了数据结构。这是一个小但关键的动作。现代 Agent 的 todo、plan、issue、workflow state 和 execution graph 都延续了同一思想：自然语言适合生成候选，结构化状态适合保持约束。

但 BabyAGI 也展示了“模型管理模型任务”的脆弱性。任务创建者可能不断生成低价值后续事项；优先级调整者可能受最近结果支配；执行结果被存储并不代表它正确；队列增长本身容易被误认为取得进展。系统优化的是“持续产生下一任务”，而不是目标是否被外部满足。

这可以抽象成第一种自治陷阱：

当 Harness 只能测量活动而不能测量结果时，Agent 会把循环存活当作任务进展。

企业运行时因此不能只记录 tool-call 数、token 数、步骤数和任务完成声明，还必须定义与业务结果相连的 evaluator。对于代码任务可能是测试、静态检查和验收条件；对于数据分析可能是查询可复现性、口径一致性和事实来源；对于业务流程则可能是外部系统状态与审批记录。

2. AutoGPT：把开放动作空间交给语言模型

AutoGPT 的早期吸引力来自更开放的循环：模型接收一个长期目标，可以选择搜索、浏览、文件、命令等动作，并用短期历史和向量记忆延续任务。与 BabyAGI 的显式队列相比，它更接近后来通用 coding agent 的体验：模型既负责局部规划，也负责选择工具和解释观察。

早期版本证明了几个方向可行：

- 模型能在多轮中保持一个粗粒度目标；
- 工具结果能作为新观察影响后续决策；
- 外部记忆可以突破单次上下文的表面限制；
- 命令与文件工具让模型从“建议者”变成“执行者”。

与此同时，开放循环把多个风险同时放大：目标漂移、重复动作、无效搜索、错误记忆、费用失控、不可逆副作用和虚假完成。模型生成一段看起来合理的自我批评，并不意味着它识别了真实错误；向量库找回语义相似内容，也不意味着内容仍然正确或适用于当前状态。

AutoGPT 的后续仓库逐步发展出平台、组件、Forge 和 benchmark 等不同项目。历史研究必须避免把这些成熟后的结构倒推到 2023 年原型上。这里讨论的是原型所代表的范式：**让模型直接拥有开放的下一步选择权，再用提示和记忆维持长期目标。**

这次实验带来的最大教训是，自治不是一个开关。至少需要拆成五个维度：

维度	问题
决策自治	下一步由模型、规则还是人决定？
工具自治	模型可以调用哪些动作？
权限自治	动作是否需要批准，能否扩大权限？
时间自治	可以持续多久，何时暂停或终止？
进化自治	能否改变记忆、技能、策略或自身 Harness？

一个系统可以拥有高决策自治，却被严格限制在只读沙箱；也可以使用固定工作流，却对某个已批准 API 拥有高工具自治。用“全自动/非自动”描述 Agent，会掩盖真正的风险边界。

3. LangChain：把 Agent 拆成可复用抽象

LangChain 在 2023 年用一套影响广泛的词汇总结当时的 Agent：Agent 是决定动作的模型，Tools 是可执行动作，Memory 负责引入过去事件，AgentExecutor 则运行循环直到满足停止条件。其典型算法直接继承 ReAct：Thought、Action、Observation 重复进行。[LangChain 2023 总结](#)

它的历史贡献在于把快速增长的模型供应商、向量库、工具和提示模式装入可复用接口。开发者不必为每个实验重新编写消息转换、输出解析和循环控制。这种“集成框架”极大降低了进入门槛，也让 Agent、Tool、Memory、Executor 成为广泛流通的工程术语。

但抽象的代价同样迅速显现。

第一，模型 API 本身快速变化。原生 function calling、结构化输出、流式事件和服务端状态不断出现，通用包装层很容易滞后或泄漏底层差异。

第二，Agent 的故障通常发生在跨层边界：提示、工具 schema、消息序列、重试、解析器和供应商响应共同作用。过深的封装使开发者看见“chain failed”，却难以复原模型究竟看到了什么。

第三，早期 memory 概念过宽。对话历史、检索知识、用户偏好、工具结果和执行状态具有不同生命周期、可信度与更新规则，把它们都称为 memory 容易制造错误抽象。

第四，生产需要的能力不只是调用组件：还包括持久化、幂等、恢复、租户隔离、审批、流式 UI、观测和评估。它们最终要求一个运行时，而不只是一个 Python 对象图。

社区中“直接调用模型 API 加一个 while loop 就够了”的反弹，在很大程度上是对抽象税和调试困难的回应。但另一面也应写清：LangChain 并没有简单消失。它后来通过 LangGraph 将重点转向持久状态、确定性与 Agent 节点混合、interrupt/resume、checkpoint 和 durable execution。官方回顾甚至明确提出，最大的竞争者始终是“不使用框架”。[LangGraph 运行时设计](#)

所以，LangChain 的历史不是“古早框架被淘汰”的单线故事，而是一次架构纠偏：

```
graph TD; A[集成与高层抽象] --> B[暴露生产问题]; B --> C[显式状态图 + 低层运行时]; C --> D[持久化、恢复、观测与部署基础设施];
```

这条路径对企业自研 Harness 很有警示意义。早期最诱人的往往是统一接口和快速 demo；长期最难替换的却是状态模型、事件协议和运行语义。平台应把稳定性投资放在后者。

4. AutoGen：把 workflow 表达成多 Agent 对话

AutoGen 将多个可定制 Agent 的对话作为应用编排机制。每个 Agent 可以组合模型、人类输入和工具，交互行为既可由自然语言也可由代码定义。[AutoGen 论文](#)

这种设计有很强的表达力。规划者、执行者、评审者和用户代理可以用统一的消息隐喻协作；新角色可以通过说明与工具配置快速加入；人类也能作为一个对话参与者插入流程。

但“万物皆消息”与“万物皆文件”一样，既是统一抽象，也是潜在陷阱。业务状态、权限决定、执行结果、取消信号和评审结论如果都退化为自然语言消息，会产生四类问题：

- 难以保证消息被恰处理好一次；
- 难以区分陈述、命令、建议和授权；
- 难以建立严格 schema 与兼容性策略；
- 难以证明终止条件和责任归属。

多 Agent 对话还容易制造“社会性拟真”：不同角色互相赞同、批评或投票，看起来像一个团队，却可能共享相同模型偏差、相同错误上下文和相同盲点。增加说话者不自动增加独立证据。

因此，多 Agent 的核心价值不是角色扮演，而是以下至少一项真实差异：

1. 不同工具或权限边界；

2. 不同上下文与信息源；
3. 不同模型能力或成本曲线；
4. 可并行的独立工作空间；
5. 独立评价标准或对抗目标。

如果这些差异不存在，单 Agent 加结构化阶段往往更便宜，也更可观测。

5. 第一次祛魅：为什么 Demo 自治不能直接进入生产

2023 年的演示通常选择开放式目标，例如“研究一个市场并建立网站”。这类目标让模型有足够空间生成令人惊喜的轨迹，却缺少可重复、可外部判定的完成条件。生产系统正相反：错误成本真实存在，输入分布不断变化，结果必须能够审计。

自主 Agent 的第一次祛魅主要来自六个错配。

5.1 语言流畅度与状态正确性错配

模型能够生成连贯的“当前进度”，但环境可能根本没有发生对应变化。Harness 必须以工具结果和外部查询维护 canonical state，而不是用模型叙述替代状态。

5.2 语义相似与记忆有效性错配

向量检索擅长找相似文本，却不负责内容真实性、时效性、权限范围或因果相关性。生产记忆需要来源、时间、作用域、置信度、失效条件和删除机制。

5.3 工具可调用与动作可授权错配

工具出现在 schema 中只说明模型知道如何提出请求，不说明它有权执行。模型选择、策略判断、用户审批与沙箱执行必须是不同步骤。

5.4 循环持续与目标进展错配

Agent 可以不断产生新的思考、搜索和任务。预算、重复检测、停滞检测和外部里程碑必须共同限制循环。

5.5 自我批评与独立验证错配

同一模型阅读自己的输出并说“看起来正确”，只能提供一种弱信号。强验证来自测试、类型系统、约束求解、外部数据、不同信息路径或人类判断。

5.6 多角色对话与多样性错配

共享模型、共享上下文和共享提示风格的多个 Agent 很可能产生相关错误。有效冗余必须测量错误相关性，而不是只计算 Agent 数量。

6. 从 Framework 到 Runtime

本轮祛魅并没有结束 Agent 方向，反而改变了工程重心。开发者开始关心：

- 状态能否持久化并从中断恢复；

- 每一步是否形成可消费的结构化事件；
- 工具执行是否幂等，失败能否安全重试；
- 人类能否在关键点暂停、修改和恢复；
- 长任务能否跨进程、跨机器和跨版本继续；
- 权限与凭证是否由模型之外的策略系统管理；
- 轨迹能否进入回放、评估和回归测试。

这些问题推动“Agent framework”向“Agent runtime”迁移。LangGraph 将开发 API 与 PregelLoop runtime 分离；OpenHands 将 Agent 控制与沙箱 Runtime 分离；Codex 用 App Server 将共享 Harness 暴露给多个客户端；DSH 则进一步把 loop、tools、session、sandbox 和 storage 都放入可组合插件系统。

Framework 主要回答“开发者怎样表达 Agent”；Runtime 还必须回答“表达出来的 Agent 如何长期、安全、可恢复地运行”。两者并非互斥，但企业平台如果只拥有前者，就会在每个应用中重复构建后者。

7. 这一代系统留下了什么

AutoGPT 和 BabyAGI 留下开放循环与任务外置；LangChain 留下 Agent/Tool/Memory/Executor 词汇和集成生态；AutoGen 留下对话式多 Agent 编排；LangGraph 则代表从高层魔法回到显式状态和持久运行语义。

它们共同证明：一个最小 Agent 的确可以很小，但一个可靠 Agent 系统不会很小。

最小 Agent：模型 + 工具 + while loop

生产 Harness：

最小 Agent

- + canonical state
- + context lifecycle
- + policy and sandbox
- + durable execution
- + observability
- + verification
- + human control
- + evaluation and governance

下一章进入 coding agent 的工程化转折。Aider 的 repository map、SWE-agent 的 Agent-Computer Interface，以及 OpenHands 的动作—观察事件流，将证明一个后来反复出现的事实：同一个模型的表现，会被它所处的接口和环境大幅改变。模型能力并不等于系统能力。

第三章 接口也是智能：Aider、SWE-agent 与 OpenHands

本章状态：正文初稿 v0.1。

2023 年的通用自主 Agent 证明模型可以循环调用工具，但没有证明它善于在大型代码仓库中工作。软件工程要求 Agent 定位相关文件、理解跨模块关系、生成能够可靠落盘的局部修改、运行测试并解释错误。模型可能知道如何写一个函数，却因看错文件、破坏补丁格式或忽略仓库约定而失败。

这一阶段最重要的发现可以概括为：

模型能力并不是系统能力的固定上限；模型看见什么、能做什么、动作如何表达、反馈怎样返回，会系统性改变任务表现。

1. Aider：上下文不是越多越好

大型仓库无法完整塞进上下文窗口。即使窗口足够大，无差别注入也会增加费用、延迟和注意力干扰。Aider 的 repository map 选择另一条路线：提取仓库中的关键符号、类型和签名，以文件依赖关系构图，再在 token 预算内通过图排序选择最相关、最重要的部分。[Aider Repository Map](#)

它实际上为模型提供了两级观察：

Repo Map：低成本、全局但有损的符号地图
Full File：高成本、局部但更完整的源码观察

模型先借助地图理解大致结构，再请求具体文件。这比预先猜测所有相关上下文更接近主动感知：Harness 提供可导航地图，模型决定何时放大细节。

这套设计包含三个可迁移原则。

第一，摘要必须服务于后续定位。一个只“概括内容”的摘要可能读起来很好，却无法帮助 Agent 找到下一步需要读取的对象。符号名、路径、签名和依赖边是可操作的索引。

第二，上下文选择需要显式预算。不同信息不是简单的“相关/不相关”，而是在有限 token、延迟和缓存条件下竞争。Harness 应把 context assembly 看成带约束优化问题。

第三，模型应能按需深化观察。一次性静态 RAG 无法预知执行过程中产生的新问题。现代 Cursor 的动态上下文、Claude Code 的搜索工具和 Codex 的文件工具都延续了这一点。

2. 编辑格式：工具接口会占用模型能力

让模型“给出正确代码”和让它“给出 Harness 能可靠应用的修改”是两个不同任务。Aider 为 whole file、search/replace block、unified diff 等多种编辑格式建立端到端 benchmark，测量代码是否正确、格式是否可解析、修改是否成功落盘并通过测试。[Aider 编辑 benchmark](#)

早期实验出现了一个反直觉结果：function calling 的结构更严格，却可能比简单文本格式表现更差。复杂格式不只增加解析约束，也占用模型用于解决代码问题的能力。不同模型对 whole、diff、diff-fenced 等格式的适应也不同。

因此，“结构化接口必然优于文本接口”不是普遍真理。正确问题是：

- 模型是否在训练或后训练中见过这种接口？
- schema 是否贴近任务的自然结构？
- 参数生成需要多少转义、定位和重复内容？
- 接口失败能否返回可修复的局部错误？
- token、延迟、正确率和修改范围之间如何权衡？

这也是模型特定 Harness 的早期证据。同一工具为不同模型提供相同 schema，看似平台中立，实际可能让某些模型承担额外认知税。企业平台需要统一语义，但不必强迫所有模型使用完全相同的表面协议；适配层可以把模型原生动作映射到统一的内部命令。

3. SWE-agent：Agent-Computer Interface 成为设计对象

SWE-agent 把接口明确命名为 Agent-Computer Interface (ACI)。它的论文结论不是只比较模型，而是说明定制 ACI 可以显著改善 Agent 浏览仓库、编辑文件、运行测试和处理程序输出的能力。[SWE-agent](#)

ACI 类似人机交互中的 UI，但用户是语言模型。好的 ACI 需要考虑模型的行为偏好和错误模式：

- 命令集合是否小而正交；
- 观察是否包含足够定位信息；
- 大输出是否截断，截断点是否可理解；
- 文件编辑是否容易精确寻址；
- 语法或 lint 错误是否及时返回；
- shell 状态是否跨步骤保持；
- 错误消息是否告诉模型怎样恢复。

传统 API 设计主要面向确定性程序：调用者会严格遵循 schema，并能用代码处理错误。ACI 面向概率性调用者：工具说明本身是模型上下文的一部分，命令命名、示例、错误文本和输出长度都会改变策略。

所以工具质量至少有四层：

Capability	是否能完成所需动作
Semantics	输入、输出和副作用是否定义清楚
Learnability	模型能否从说明与反馈学会正确使用
Governance	动作能否授权、审计、隔离和撤销

很多 MCP 或内部工具只完成第一层：接口“能调用”，却没有为 Agent 的可学习性与企业治理设计。

4. CodeAct：代码作为动作语言

JSON tool call 将动作限制为预定义函数及其参数。CodeAct 提出用可执行 Python 作为统一动作空间，使模型可以组合库调用、中间计算和控制流，并根据执行观察继续修订。[CodeAct](#)

代码动作的优势来自组合性。假设 Agent 需要读取数据、过滤、聚合并画图。函数调用模式可能产生多轮 payload 往返；代码模式可以在沙箱内完成多个步骤，只把必要结果送回模型。

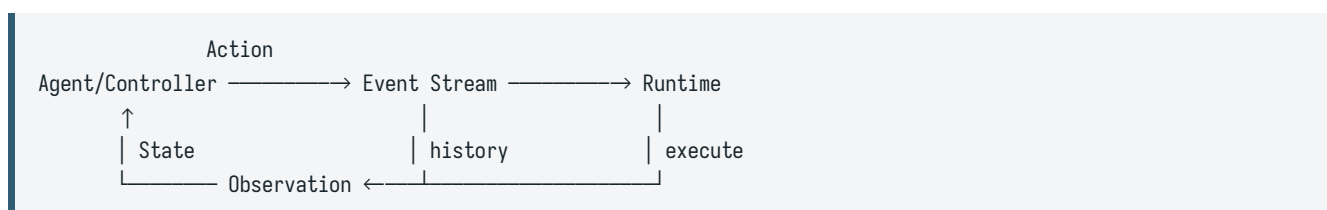
但代码动作也扩大了风险面：

- 动作空间从有限 schema 扩大为通用程序；
- 静态权限判断更困难；
- 中间状态可能留在进程、文件或网络中；
- 可重试性和幂等性更难保证；
- 执行日志可能泄露秘密或产生巨大输出。

因此 CodeAct 不是 function calling 的普遍替代，而是一种适用于高组合性任务的 ACI。一个成熟 Harness 可以同时提供：高风险业务动作使用窄 schema 工具；数据转换和探索在受控代码沙箱中完成；确定性重复流程编译成普通程序或工作流。

5. OpenHands：把 Agent 与 Runtime 分开

OpenHands 将系统拆成三个核心概念：Agent 根据状态产生 Action；Event Stream 按时间保存 Action 与 Observation；Runtime 在沙箱环境中执行 Action 并返回 Observation。[OpenHands 论文](#)



这个分离具有深远的工程意义。

Agent 是决策面：它可以替换模型、提示和策略。Runtime 是执行面：它负责进程、文件、浏览器和网络等真实副作用。Event Stream 则成为二者之间可追溯的协议事实。

如果 Agent 进程崩溃，Runtime 不一定必须销毁；如果 UI 断开，事件仍可持久化；如果要回放问题，可以重建动作—观察历史；如果要并行评估，同一 Agent 可以连接多个隔离 Runtime；如果要支持远程执行，控制面不必进入容器。

OpenHands 的 Docker Runtime 在用户镜像中加入 action-execution server，通过客户端—服务器接口发送动作和接收观察。这种结构把任意代码执行放入独立安全域，也让本地 Docker、远程容器和托管沙箱可以实现同一 Runtime 契约。[Runtime Architecture](#)

6. Event Stream 不等于完整事件溯源

按时间记录 Action 与 Observation 很有价值，但不能看到“event stream”就自动假设系统满足严格事件溯源语义。企业实现还要回答：

- event 是否不可变，允许怎样的脱敏和删除？
- 每个 event 是否有稳定 id、因果 id 和幂等键？
- schema 如何版本化？旧 consumer 如何升级？
- 大型输出存正文还是对象存储引用？
- secret 在写日志之前还是之后脱敏？
- 并发工具和 subagent event 如何排序？
- projection 出错后能否从日志重建？
- 用户数据删除与不可变审计如何协调？

因此，动作—观察日志是基础，不是终点。参考架构将在后文区分原始 execution event、面向模型的 observation、面向 UI 的 projection 与面向审计的 security record。它们可能源自同一动作，却有不同数据保留和访问策略。

7. Benchmark 开始评价“模型 × Harness”

Aider 的 benchmark 同时评价模型解题和编辑格式；SWE-agent 比较 ACI；OpenHands 将 Agent 和 Runtime 接入多个软件工程 benchmark。这意味着评价单位开始从裸模型转向系统组合。

可以将任务成功率表示为：

```
Success = f(Model, Harness, Environment, TaskDistribution, Budget)
```

如果只报告模型名称而不固定 Harness、环境镜像、工具版本、token 预算和重试策略，结果无法归因。反过来，如果 Harness 更新后分数提高，也不能立即说 Harness 本身更好：可能是评测污染、任务方差、模型后端变化或预算增加。

这为本书后面的 2×2 Model × Harness 评估奠定基础：固定任务集，交叉运行旧/新模型与旧/新 Harness，多次采样，并同时测量成功率、成本、延迟、安全违规和人工介入。只有这样才能区分模型收益、Harness 收益与交互效应。

8. 从接口工程得到的七条原则

这一阶段留下七条稳定原则：

1. 全量上下文不是默认答案；应提供可导航的低成本地图和按需深化机制。
2. 工具接口是模型行为的一部分，必须用目标模型做端到端评估。
3. 表面协议可以模型特定，内部动作语义必须稳定。
4. 通用代码动作适合组合计算，但必须进入更强的沙箱与审计域。
5. 决策控制面与副作用执行面应具有清晰协议边界。

6. Action/Observation 应成为结构化、可关联、可回放的运行时事实。

7. 评估对象应是模型、Harness、环境、预算与任务分布的组合。

这些原则解释了为什么后来的 Claude Code、Codex、Cursor 不只是“聊天框加 shell”。它们在上下文、编辑、命令、权限、会话和验证上各自选择了不同 ACI。下一篇将不再按时间讲故事，而是拆开现代 Harness 的核心责任，建立一套可用于产品分析和自研设计的统一模型。

第四章 Coding Agent 转折：真实仓库成为反馈环境

本章状态：正文初稿 v0.1。

2023 年的通用 Agent 热潮证明了模型可以循环调用工具，却没有证明它能稳定完成真实工作。Coding Agent 改变了问题形态：仓库提供持久状态，编译器、测试和版本控制提供外部反馈，patch 提供可审查的交付物。Harness 从“让模型继续思考”转向“让模型在环境里形成可验证闭环”。

1. 从聊天记录到工作树

早期框架倾向把任务状态放在消息、计划列表或向量记忆中。软件工程的权威状态却在文件系统、Git、依赖环境和测试结果里。模型不必在上下文复述全部代码，只需要能搜索、定位、修改、执行和回读。这一变化奠定了现代 Harness 的基本形态：薄决策循环，厚环境适配。

Aider 的 repository map 用代码图和 token budget 选择相关符号，使模型在有限上下文中获得仓库结构，而不是粗暴注入所有文件。[Aider Repository Map](#) SWE-agent 进一步提出 ACI，即 Agent-Computer Interface：命令、观察格式和反馈设计会显著影响同一模型的解题能力。[SWE-agent](#)

这说明接口不是模型之外的包装。一个返回数万行终端噪声的 shell，与一个保留退出码、截断策略、错误定位和可追溯 artifact 的 shell，对模型来说是两个不同环境。

2. 可执行反馈改变了规划

传统规划假设动作模型相对明确；真实仓库中的依赖、隐含约束和测试常常只有执行后才暴露。现代 coding loop 因而更像在线控制：观察局部状态，提出最小变更，运行检查，根据误差修正。

```
issue → inspect → hypothesis → patch → test
                ↑           |
                └─ diagnose ─┘
```

成功不再取决于一次生成完整方案，而取决于 Harness 是否让失败变得可诊断。命令超时、测试失败、环境损坏和权限拒绝必须具有不同语义；否则模型会把基础设施问题误解为代码问题。

3. CodeAct 与统一动作空间

CodeAct 研究表明，让 Agent 通过可执行代码组织复杂工具交互，可以减少大量逐个 JSON tool call 的往返，并利用编程语言的循环、变量和组合能力。[Executable Code Actions](#) 这条路线后来体现在 Code Mode、sandbox script 和“计算留在环境、只把结果带回上下文”等设计中。

但统一 shell 或代码动作也扩大了权限面。表达力越强，越需要沙箱、网络策略、凭证代理和确定性审计。Coding Agent 的历史因此同时推动了自治与约束。

4. OpenHands 的分离

OpenHands 将 Agent 的决策与 Runtime 的执行显式分开，通过动作、观察和事件流连接；Runtime 可以是 Docker 或远程执行环境。[OpenHands Runtime Architecture](#) 这种分离使模型策略可以替换，环境生命周期、隔离和日志则由另一层负责。

它对企业架构的启示不是必须采用其类结构，而是：不让模型循环直接持有宿主进程。Agent 产生规范化 action，Runtime 验证并执行，observation 再成为下一步输入。由此可以在协议边界插入策略、录制、重放和模拟器。

5. Benchmark 的双重作用

SWE-bench 把真实 GitHub issue、仓库和测试组成可执行评测，使模型与 Harness 的组合成为测量对象，而非只测代码补全。[SWE-bench](#) 它推动工具、搜索、编辑、上下文和验证快速演化，也暴露一个事实：排行榜分数同时包含模型、Harness、环境构建和评测质量，不能简单归因给模型。

后来 SWE-bench Verified 的修订与退役进一步表明，环境反馈虽比文本 judge 更硬，也不是绝对真理。测试可能错误，公开样本可能污染，Agent 可能投机。Coding Agent 只是提供了更好的实验场，而不是自动解决验证问题。

6. 现代 Harness 的形成

从 Aider、SWE-agent、CodeAct 和 OpenHands，可以看见五个后来成为主流的原则：上下文按需编译；工具接口针对模型优化；执行发生在可控环境；每个动作产生结构化观察；完成由外部证据决定。Claude Code、Codex、Cursor 等产品的差异主要发生在这些原则的工程取舍，而不是是否拥有一个 `while` 循环。

Coding Agent 转折的真正意义，是把 Agent 从语言产品变成运行时问题。模型仍负责提出高熵决策，但文件、进程、权限、测试、状态和提交由软件系统承载。Harness 由此成为模型与真实世界之间的责任边界。

第二篇 原理：生产级 Harness 的构成

第五章 Agent = Model × Harness × Environment × Feedback

本章状态：正文初稿 v0.1。章节编号依照全书目录；第四章将在历史资料补齐后撰写。

“Agent = Model + Harness”是一条有用的传播公式，但对企业架构仍然太粗。它容易让人把环境、验证与反馈也塞进 Harness，最终得到“除模型外一切都是 Harness”的不可操作定义。

本书采用一个乘法式系统模型：

```
Agent System Capability
= Model × Harness × Environment × Feedback
```

乘号表达的不是精确数学关系，而是互相制约：任何一项接近零，系统能力都会大幅下降。优秀模型放进贫乏工具和错误权限中无法完成任务；优秀 Harness 不能让模型解决超出其理解边界的问题；不可复现的环境会让正确计划执行失败；没有外部反馈的系统无法区分“生成了结果”和“结果真的有效”。

1. Model：概率性策略与生成器

模型接收有限上下文，输出文本、结构化动作或代码。它擅长：

- 从非结构化目标中提出解释和候选计划；
- 在不完整信息下选择下一观察或动作；
- 生成代码、查询、文档和工具参数；
- 根据错误反馈诊断并修改方案；
- 在多个候选之间做语义比较。

模型不天然拥有持久状态、真实权限、可靠时钟、事务语义和外部世界真值。即便 API 提供 conversation id 或服务端工具，这些也是服务在模型之外提供的能力。

架构上应把模型看成一种概率性策略：

```
proposal ~ Model(context, action_space, sampling_policy)
```

它提出下一步，而不是直接产生已授权副作用。这个区分让平台能够在模型与环境之间插入验证、策略和审批。

2. Harness：运行时中介与控制系统

Harness 负责把目标、模型和环境组织成可持续执行的任务。综合现代产品与研究，本书把其责任归为十个域：

1. **任务契约**：目标、范围、约束、交付物、完成证据；
2. **循环控制**：step、turn、retry、budget、stop、cancel；

3. **上下文生命周期**：选择、排序、缓存、压缩和重新发现；
4. **工具与动作协议**：schema、调用、结果、错误、版本和发现；
5. **状态与记忆**：会话、执行、任务、经验和长期资产；
6. **策略与授权**：身份、权限、审批、凭证和数据范围；
7. **执行协调**：Runtime 分配、并发、超时、恢复和隔离；
8. **验证与完成**：测试、evaluator、证据包和停止门禁；
9. **观测与归因**：事件、trace、成本、失败分类和人工介入；
10. **评估与进化治理**：回放、回归、实验、发布、回滚和审计。

“AI Harness Engineering”研究同样主张能力来自 model-harness-environment system，并列出了任务说明、上下文、工具、记忆、任务状态、可观测性、失败归因、验证、权限等责任。[AI Harness Engineering](#)

Harness 不必在一个进程或代码库中实现。桌面客户端、App Server、策略服务、沙箱调度器、事件存储和评估平台可以共同构成逻辑 Harness。判断边界的关键不是部署拓扑，而是谁拥有运行语义。

3. Environment：不是一个 shell，而是任务世界

环境包含 Agent 可以观察和改变的外部状态：代码仓库、文件系统、容器、浏览器、数据库、SaaS API、CI、日志、指标以及人类组织。

企业设计常犯的错误，是把“给 Agent 一个终端”当作完成环境建设。Shell 确实是高度组合的通用接口，但环境还需要：

- 可复现的依赖和初始化；
- 与任务对应的身份和网络范围；
- 可观测的应用、日志和指标；
- 快照、fork、清理和资源配额；
- 稳定的服务发现和秘密注入；
- 任务完成后可查询的权威状态。

OpenAI 的 Harness 工程实践把每个 worktree 的应用、浏览器、日志和指标都暴露给 Agent，使其能够复现和验证，而不仅是修改源码。[OpenAI Harness Engineering](#)

环境可读性是被低估的能力杠杆。与其反复提示模型“务必检查启动耗时”，不如让它能够查询启动 trace；与其让模型猜测页面是否正确，不如提供 DOM、截图和浏览器交互；与其把数据库错误复制进 prompt，不如提供只读诊断工具和明确 schema。

4. Feedback：观测不等于评价

工具返回 stdout 是 observation，但不一定是 feedback。Feedback 指能改变系统对“这一步或这次任务有多好”的判断信号。

可以分成四类：

类型	例子	主要用途
执行反馈	exit code、异常、HTTP 状态	即时修复动作
任务反馈	测试、验收规则、业务结果	判断是否完成
人类反馈	批准、修改、拒绝、偏好	处理价值与需求判断
群体反馈	线上指标、回归集、事故、成本	更新 Harness、技能或模型

反馈必须尽可能靠近真实目标。单元测试通过可能仍破坏用户流程；人工点“接受”可能只是没时间审查；模型 judge 的高分可能来自提示泄漏。可信系统需要多信号组合，并记录每个信号的来源和局限。

5. 为什么使用乘法而不是加法

假设两个团队使用同一模型。团队 A 提供快速但不稳定的环境、数十个含糊工具和“模型说完成即完成”；团队 B 提供小而清晰的工具集、可复现 workspace、外部测试和失败恢复。二者不是在相同 Agent 上增加少量功能，而是在构建不同能力的系统。

乘法视角带来三项决策变化。

第一，模型升级不能跳过系统回归。更强模型可能更频繁使用工具、生成更长命令或绕过旧 guardrail，导致安全和成本退化。

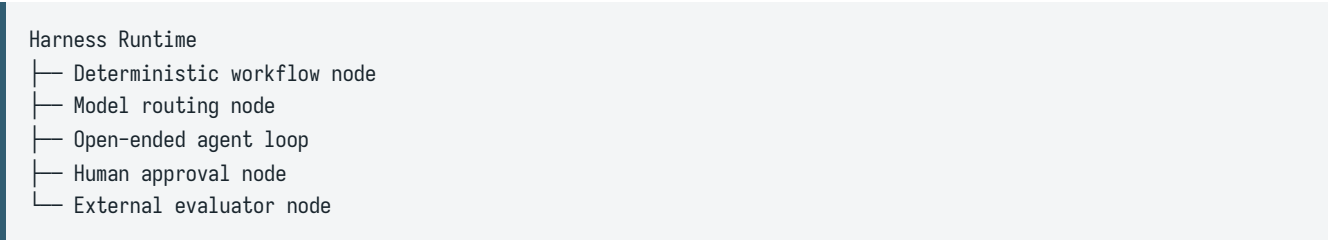
第二，Harness 优化必须跨模型验证。某个为模型 A 设计的提示、工具格式或压缩策略可能损害模型 B。

第三，环境与反馈是产品能力，不是“运维配套”。如果 Agent 无法观察真实系统和验证结果，它的自治上限不会因模型参数增加而自然消失。

6. Workflow、Agent 与 Harness 的关系

Anthropic 将 workflow 与 agent 区分：workflow 通过预定义代码路径编排模型和工具；agent 则让模型动态决定过程与工具使用，并建议优先使用简单、可组合模式。[Building Effective Agents](#)

Harness 可以同时运行两者：



企业系统不应把“更 agentic”当成天然更先进。稳定、重复、高风险流程应尽量确定化；只有步骤无法事先穷举、需要语义判断或探索时，才扩大模型决策空间。

一个实用判据是：如果下一步能由便宜、稳定、可测试的代码决定，就不应消耗模型不确定性预算。模型应集中处理无法可靠编码的判断。

7. 控制面、数据面与执行面

参考架构将 Harness 拆成三个逻辑平面。

7.1 控制面

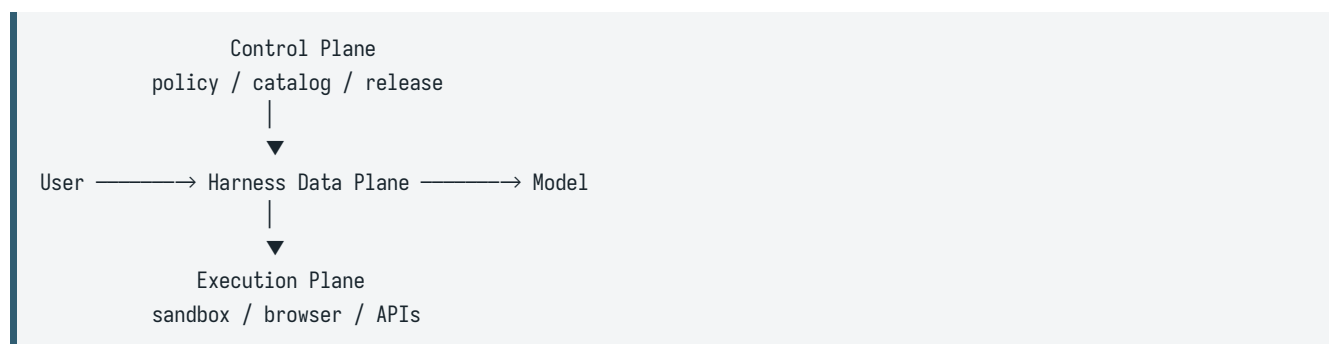
管理配置、身份、策略、模型目录、工具目录、技能版本、实验、租户和发布。控制面决定“什么可以被运行”，但不进入每一步高频数据路径。

7.2 数据面

承载会话、事件、上下文、模型调用、工具请求、状态 projection 和实时交互。它决定“一次任务如何推进”。

7.3 执行面

运行命令、代码、浏览器和外部连接，承载真实副作用。它决定“动作在哪里、以什么权限发生”。



分平面不是为了追求微服务数量，而是建立不同信任边界。允许 Agent 修改数据面的临时计划，不代表允许它修改控制面的根权限；允许执行面持有短期凭证，不代表模型上下文可以读取凭证值。

8. 概率性建议与确定性约束

企业 Harness 最重要的设计原则之一，是区分“希望模型遵守”与“系统保证不会违反”。

系统提示中的“不要访问生产数据库”是概率性建议；网络策略和身份权限才是确定性约束。提示中的“修改前请询问”可能被误解；工具调用前的审批状态机才提供可审计保证。

可以把约束按强度排列：

```
建议: prompt / tool description
引导: workflow / mode / model routing
检查: validator / policy decision
限制: capability / IAM / network policy
隔离: container / VM / separate account
证明: external verification / signed audit
```

越靠近不可逆副作用，越应使用后面的机制。Prompt 仍然重要，因为它减少无效尝试和审批噪声，但不能承担安全根信任。

9. Harness 的厚与薄

现代产品存在明显分歧。DSH 主张一切皆插件，Codex 建立丰富核心与协议服务器，Cursor进行模型特定调优；Pi 则刻意保持四工具核心，不内置 MCP、subagent、plan mode、permission popup 和 background bash，把这些交给容器、tmux、技能或扩展。

“厚 Harness”通常提供一致体验、治理和开箱能力，却增加核心复杂度、模型耦合和升级风险。“薄 Harness”更透明、更容易理解和组合，却把安全与运维责任交给部署者。

判断能力是否应进入核心，可以问五个问题：

- 1. 是否影响安全或状态正确性？
- 2. 是否需要跨所有 Agent 保持一致语义？
- 3. 是否位于恢复、取消或审计关键路径？
- 4. 是否必须在模型不可用时仍然工作？
- 5. 是否能由普通环境工具以同等可靠性提供？

前四项多为“是”时倾向核心；第五项为“是”且风险较低时倾向扩展。计划展示可以是插件，权限执行不能只靠插件约定；todo UI 可以替换，事件 idempotency 应进入底层协议。

10. 一个可实现的最小分层

对自研企业平台，建议从六层开始，而不是一次实现所有产品功能：

L6 Experience	CLI / IDE / Web / API / automation
L5 Agent Patterns	loop / workflow / multi-agent / evaluator
L4 Runtime State	task / session / event / context / memory
L3 Governance	identity / policy / approval / audit
L2 Execution	sandbox / tools / browser / connectors
L1 Model Gateway	provider / routing / cache / quota

每层只依赖下层稳定契约。Experience 不直接执行 shell；Agent pattern 不直接读取生产凭证；模型 gateway 不负责业务完成判断；execution 不解释自然语言意图。

这套分层仍允许单体实现。早期平台可以在一个进程中部署，但接口和状态所有权应从一开始分清，否则后续多租户、远程沙箱和多客户端接入会迫使系统整体重写。

11. 本书的统一分析模板

后续每个产品案例都使用同一组问题：

- 1. 核心 loop 和停止语义是什么？
- 2. 上下文如何装配、缓存、压缩和恢复？
- 3. 工具如何描述、发现、执行和返回错误？
- 4. 状态和记忆存在哪里，生命周期如何？
- 5. 权限、审批、凭证和沙箱如何分层？

6. Agent 如何验证结果并声明完成？
7. 多 Agent 如何隔离、委派、取消和合并？
8. 客户端与 Harness 通过什么协议交互？
9. 运行轨迹如何观测、评估和归因？
10. 哪些部分可扩展、可替换或可进化？

这可以避免产品比较退化为功能勾选。两个产品都支持 “subagent”，其委派语义可能完全不同；两个产品都支持 “sandbox”，一个可能只是默认限制，另一个可能具有企业策略和临时扩权；两个产品都支持 “memory”，保存的可能分别是聊天摘要、用户偏好或可执行技能。

本章得到的核心结论是：Harness 不是提示词集合，也不只是 while loop。它是模型与环境之间负责运行语义、信任边界和证据闭环的控制系统。下一章将把最核心的 agent loop 展开为状态机，讨论 turn、step、stream、cancel、retry、compaction 和 crash recovery 如何共同决定长任务是否真正可运行。

第六章 Agent Loop：从 while 循环到持久状态机

本章状态：正文初稿 v0.1。

几乎所有工具型 Agent 都能用十几行伪代码表达，但生产故障很少发生在那十几行的正常路径。真正困难的是：并行工具只完成一半怎么办？用户在命令运行期间取消怎么办？模型返回 final answer 是否意味着任务完成？进程在外部副作用提交后、结果落库前崩溃怎么办？

因此，企业 Harness 不应把 loop 只实现为内存中的 `while`，而应把它设计成有持久身份、明确状态、可恢复转移和副作用账本的状态机。

1. Thread、Turn、Step 与 Attempt

现代产品对术语并不完全一致。本书采用四级执行单位：

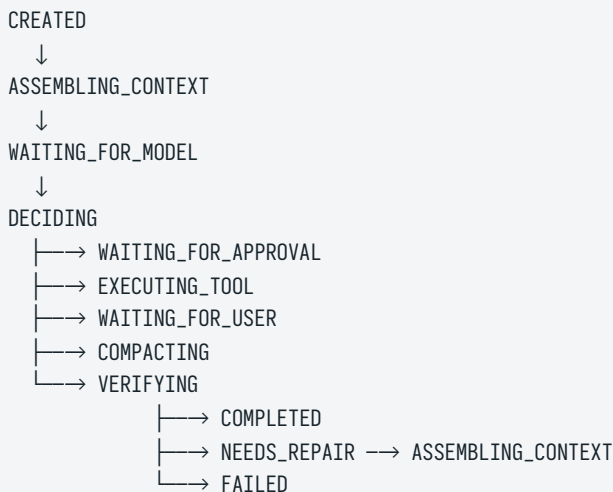
- Thread 围绕一个持续协作上下文的会话
 - Turn 用户或外部事件触发的一次控制权往返
 - Step 一次模型决策或工具执行等可观测阶段
 - Attempt 某个 Step 的一次具体尝试

Codex 将用户输入到最终 assistant message 的过程称为 turn，其中可以包含多次模型推理与工具调用。[Codex Agent Loop](#) Claude Agent SDK 也循环执行“评估—工具—结果”，直到模型输出不含工具调用的响应，并另外返回包含 token、成本和 session id 的结果消息。[Claude Agent Loop](#)

区分 Step 与 Attempt 是为了安全重试。网络超时后再次调用同一工具，是同一个逻辑 Step 的新 Attempt；模型看到错误后改用另一种方案，则通常是新 Step。如果两者混淆，成本、归因和幂等判断都会失真。

2. 最小状态机

一个可运行的 turn 至少包含这些状态：



任意活动状态 → CANCELLING → CANCELLED
任意活动状态 → SUSPENDED → 恢复点

FAILED、CANCELLED 和 COMPLETED 必须区分。用户取消不代表模型失败；预算耗尽也不代表业务任务不可完成；工具拒绝可能是策略结果而非技术错误。状态影响能否重试、是否收费、如何告警以及下次恢复时模型应看到什么。

3. 模型停止不等于任务完成

模型输出无 tool call 的 assistant message，通常结束当前 loop。但它可能是在提问、报告阻塞、拒绝任务，或者错误地声称完成。

所以至少需要两个判定：

model_stop	模型本轮不再请求动作
task_completion	外部完成契约已经满足

Harness 可按任务类型选择完成策略：

- 对话问答：final response 可能足够；
- 代码修改：要求工作区有预期 diff，并运行指定测试；
- 数据任务：要求产生可复现查询、结果和口径说明；
- 高风险业务动作：要求外部系统确认与审计事件；
- 研究任务：要求来源覆盖、引用验证和未知项披露。

如果验证失败，系统应把结构化差异作为新 observation 送回 Agent，而不是只说“再检查一下”。

4. Streaming 是事件协议，不只是打字动画

长任务需要实时展示模型文本、工具请求、命令输出、审批和状态变化。若 streaming 只实现为不可恢复的 socket 文本，断线后客户端无法知道遗漏了什么。

建议每个流事件包含：

```
event_id
thread_id / turn_id / step_id / attempt_id
sequence_or_cursor
event_type
timestamp
payload_ref
causation_id / correlation_id
schema_version
visibility_class
```

客户端用 cursor 重连，服务端重放缺失事件；大 payload 存对象存储，只在事件中保留 hash、类型和访问引用；同一原始事件可以投影为模型 observation、用户 UI 和审计记录，但必须遵循不同脱敏策略。

5. 取消必须贯穿调用链

用户点击停止后，仅停止下一次模型调用是不够的。正在运行的 shell、浏览器任务、subagent 和远程 job 仍可能继续产生副作用。

取消应沿所有权树传播：

```
Turn cancellation
├─ model request cancellation
├─ active tool cancellation
│   └─ process group termination
│       └─ remote job cancel request
├─ child agent cancellation
└─ pending approval invalidation
```

取消还存在竞态：动作可能在取消到达前已经提交。状态不能简单写成“cancelled, nothing happened”，而应记录 `cancel_requested_at`、`effect_committed_at` 和最终 reconciliation。对不可撤销动作，应返回“取消请求已接收，但动作已提交”的明确结果。

6. Timeout、Retry、Resume 不是一回事

Timeout 是 Harness 不再等待某个 Attempt；Retry 是重新尝试逻辑 Step；Resume 是从持久执行状态继续整个任务。三者不能互换。

安全重试要求工具具备以下一种语义：

1. 天然幂等，例如读取文件；
2. 接受 idempotency key，由下游去重；
3. Harness effect ledger 能确认之前未提交；
4. 有明确 compensation，可撤销重复效果；
5. 无法自动判断，转人工 reconciliation。

AWS Durable Execution 文档把幂等定义为重复运行仍产生同样效果，并强调执行确认和 checkpoint 对重试语义的重要性。[Idempotency and Retries](#)

模型调用通常可以重试生成，但结果并不相同；读工具可以重试；发送邮件、付款和合并代码不能因网络超时而盲目重试。

7. 副作用账本与不确定提交

最危险的崩溃窗口是：外部动作已执行，但 Harness 尚未保存结果。

```
persist INTENT
authorize exact action
execute external effect
persist OUTCOME
```

如果在第三、四步之间崩溃，恢复时只知道 intent，不知道 effect 是否发生。这是分布式系统中的不确定提交，不是多问模型一次就能解决。

Effect ledger 至少记录：目标系统、动作类型、规范化参数 hash、idempotency key、授权依据、开始时间、下游 request id、已知结果和 verification status。恢复器优先向下游查询，而不是重新发送。

8. Checkpoint 应保存什么

仅保存聊天历史不足以恢复执行。Checkpoint 应覆盖：

- 当前 thread/turn/step/attempt 状态；
- canonical task state 与未满足验收条件；
- 已装配上下文的版本或可重建引用；
- 模型、工具、技能、策略和环境版本；
- 已提交与未确定的副作用；
- 活动 child agent 和 remote job handle；
- budget 使用量与剩余额度；
- 当前 workspace snapshot/commit/image 标识。

Claude Code 把消息、工具调用和结果写入 JSONL，从而支持 resume、rewind 和 fork；这对个人会话很有效。[Claude Code Sessions](#) 企业平台还需要数据库一致性、租户隔离、加密、保留策略和跨版本迁移。

9. Compaction 是有损状态迁移

上下文接近上限时，Claude SDK 与 Codex 都会压缩历史。Codex 强调缓存依赖精确前缀匹配，并尽量通过追加消息表达中途配置变化；其服务端 compaction 以较短 items 替代旧 input。[Codex Agent Loop](#)

压缩不是普通摘要，而是一次有损状态迁移。至少要保护：

- 用户目标和后续修订；
- 权限与安全约束；
- 已验证事实及来源；
- 未完成承诺和阻塞；

- workspace 关键变化；
- 失败尝试及不要重复的原因；
- 可重新发现原始记录的指针。

最佳实践是“双轨状态”：模型上下文可以压缩，canonical execution state 不随摘要丢失。模型需要细节时，通过事件、文件或 artifact 重新读取。

10. Error Taxonomy 决定恢复策略

错误不应全部变成一段 tool result。建议至少分类：

类别	例子	默认策略
Model transient	限流、网断	退避重试/切换
Model semantic	无效工具参数	结构化反馈给模型
Policy denied	权限不足	请求批准或改变方案
Tool transient	服务超时	幂等重试
Tool deterministic	参数非法、文件不存在	修正输入
Environment drift	依赖/分支变化	重新观察或重建环境
Verification failure	测试失败	进入 repair loop
Budget exhausted	token/时间/费用耗尽	暂停并报告
Unknown effect	提交状态不明	reconciliation/人工

错误类型必须由确定性层尽可能识别。让模型从任意 stderr 猜测“要不要重试”会造成 retry storm 和重复副作用。

11. 并行工具与一致性

模型可能一次请求多个工具。只有互相独立的只读观察适合默认并行。多个写操作可能基于同一旧状态，产生冲突。

Harness 可在执行前计算 action footprint：读取集、写入集、外部目标、凭证域和工作区。若 footprint 重叠，则串行执行、隔离到不同 workspace，或使用乐观并发控制并在提交时检查版本。

同理，多 Agent 并行不应共享未经协调的可变目录。Codex 使用 worktree 隔离不同线程，是把冲突从任意文件覆盖转化为显式合并问题。

12. 一个更完整的循环伪代码

```
function run_turn(turn_id):
    restore_or_create_state(turn_id)

    while not terminal(state):
        enforce_budget_and_cancellation(state)

        if context_needs_compaction(state):
            compact_model_view_preserving_canonical_state()

        proposal = call_model(build_context(state))
        persist(proposal)

        if proposal.requests_actions:
            for action in plan_execution(proposal.actions):
                validate_schema(action)
                decision = authorize(action, current_state)
                if decision.requires_human:
                    suspend_with_durable_approval_request()
                else:
                    execute_with_effect_ledger(action)
            continue

        verification = evaluate_completion_contract(state, proposal)
        if verification.passed:
            complete_with_evidence_package()
        elif verification.repairable:
            append_structured_feedback(verification)
        else:
            fail_or_escalate(verification)
```

Loop 的复杂度并不在 `while`，而在每个函数都跨越状态、权限和失败边界。

13. 最小可靠性测试集

任何企业 Harness 在上线前都应自动执行这些“杀进程”测试：

1. 模型返回工具调用后立即崩溃；
2. 工具已提交副作用、结果落库前崩溃；
3. 并行工具一个成功一个超时；
4. 等待审批时服务重启；
5. 用户取消与动作提交同时发生；
6. compaction 前后继续同一任务；
7. 恢复时模型、工具或策略版本已改变；
8. child agent 完成但 parent 丢失连接；
9. streaming 客户端断线后按 cursor 重连；
10. 相同 idempotency key 被并发提交。

如果平台只通过正常路径 demo，而没有这些故障注入，它拥有的是可演示 loop，不是持久运行时。

本章的结论是：Agent turn 必须拥有独立于模型上下文和进程生命周期的持久身份；副作用必须有可查询账本；停止必须经过外部完成契约。下一章将专门讨论上下文系统，因为长任务的另一个核心难题不是“存下所有历史”，而是让模型在正确时刻看到正确、可信且足够少的信息。

第七章 上下文、缓存、压缩与记忆

本章状态：正文初稿 v0.1。

模型在一次推理中只能依据当前上下文行动。对 Harness 而言，“记住一切”并不是目标；目标是在正确时刻，把可信、相关、足够且成本可接受的信息放到模型可见位置，同时保留原始事实以供重新发现。

上下文系统最常见的设计错误，是把对话历史、任务状态、知识检索、用户偏好、长期经验和可执行技能都塞进一个名为 memory 的容器。它们的信任等级、生命周期和更新权限完全不同。

1. 五种必须分开的信息

Working Context	当前模型请求实际看见的内容
Conversation Log	用户、模型、工具交互的原始历史
Canonical State	任务、约束、执行与副作用的权威状态
Long-term Memory	跨会话复用的事实、偏好和经验
Skills/Artifacts	可执行程序、流程、模板和文档资产

Working Context 是临时编译产物，可以被压缩和重排；Conversation Log 用于审计、恢复和重新发现；Canonical State 不应依赖模型摘要保持正确；Long-term Memory 必须有来源与失效策略；Skills 则需要版本、权限和供应链治理。

如果把这五类混在一起，压缩可能删除任务约束，模型写入的猜测可能变成“长期事实”，删除聊天记录可能意外删除审计状态，而一条未经审查的 memory 甚至可能在未来会话中持续注入恶意指令。

2. Context Assembly 是一次编译

每次模型调用前，Harness 应从多个来源编译上下文：

```
Context = compile(
    system_contract,
    policy_view,
    task_state,
    recent_events,
    retrieved_knowledge,
    active_skills,
    tool_catalog,
    environment_snapshot,
    token_budget
)
```

“编译”意味着过程可重复、可观测、可测试。每个片段应记录来源、版本、token 数、选择理由和可见性。出现错误时，工程师能够回答模型究竟看见了哪一版规则、哪些文件、哪些记忆，而不是只保存最终 prompt 文本。

Context compiler 还应处理冲突优先级。组织策略、项目规则、用户本轮要求、旧记忆发生冲突时，不能依赖它们在 prompt 中的偶然位置决定结果。平台应先在确定性层识别冲突，再向模型呈现明确、最小的有效约束。

3. 长窗口不是无限注意力

“Lost in the Middle” 研究发现，长上下文模型对信息位置敏感，相关内容位于中部时性能可能显著下降。[Lost in the Middle](#)

这不意味着所有现代模型都以相同程度失败，但它推翻了“只要窗口放得下，就等于模型能同等利用”的假设。长上下文还带来成本、延迟、缓存失效和互相矛盾信息增加。

上下文设计因此需要四种预算：

- 容量预算：窗口最多容纳多少；
- 注意力预算：模型能否稳定利用；
- 经济预算：输入与 cache read 成本；
- 变化预算：哪些片段变动会破坏前缀缓存。

相关性不是唯一排序信号。任务契约和安全约束即使语义上不接近当前动作，也必须保留；最近错误可能比历史上更相似的成功案例更重要；已经失效的记忆即使高度相似也应排除。

4. 静态上下文与动态发现

静态上下文每次调用都加载，适合短小、稳定、高频使用的信息，例如当前目录、关键任务契约和少量项目规则。动态上下文由模型通过文件、搜索或工具按需获取，适合大型、低频或快速变化的信息。

Cursor 公开描述了从大量静态注入转向动态发现的路径：长工具输出写入文件，摘要后保留历史文件引用，MCP 工具描述按需读取，终端输出同步到文件系统供 grep。[Dynamic Context Discovery](#)

文件在这里不是万能答案，而是简单、可寻址、可分页、可搜索的外部存储抽象。它让 payload 不必永久占据 prompt，也让模型能重新读取原始证据。

一个实用策略是：

Always-on: 任务契约、关键策略、当前状态摘要
Index: 文件地图、技能目录、工具目录、记忆索引
On-demand: 原始文件、日志、历史、完整工具 schema
Pinned: 本轮验证所需证据与未解决错误

5. Prompt Cache 是架构约束

缓存命中通常要求请求拥有相同前缀。Codex 公开说明，工具顺序变化、模型变化、沙箱或工作目录变化都可能导致 cache miss，因此尽量保持静态内容在前，并把中途配置变化追加成新消息，而不是修改旧前缀。[Codex Agent Loop](#)

这意味着 context assembly 不仅优化语义，也优化布局稳定性：

```
[stable system + stable tools + stable project rules]
[session history with append-only changes]
[latest dynamic observations]
[current user request]
```

工具目录来自动态 MCP 时，必须稳定排序并谨慎处理 `tools/list_changed`。否则一次无关的工具发现变化可能让长会话失去缓存收益。

缓存不能影响正确性。Harness 不应为了 cache hit 隐瞒已变化的权限或环境；正确做法是保留旧前缀并追加明确的状态更新，同时在确定性策略层立即生效。

6. Compaction 是有损编译，不是删除旧消息

当上下文逼近阈值，Harness 可将旧历史压缩为结构化摘要。Claude Code 会发出 compaction boundary；Codex 使用服务端 compact 结果替换较长输入。Claude Code 还会在压缩后重新注入系统 prompt、根级项目规则和 auto memory，而路径规则需要在再次读取相关文件后恢复。[Claude Context Window](#)

压缩摘要至少应包含：

```
Goal and acceptance criteria
Confirmed constraints and approvals
Observed facts with source pointers
Workspace/environment changes
Decisions and rationale
Failed attempts and why
Open questions and blockers
Active plan and next action
Pointers to raw history/artifacts
```

压缩前后应运行 continuity checks：目标是否保持、权限是否保持、未完成项是否保持、关键 artifact 是否仍可定位。对于高风险任务，可把 compaction 当成 checkpoint，生成 hash 和差异报告。

反复压缩摘要会累积失真。建议始终从原始事件和 canonical state 生成新摘要，而不是只压缩上一次摘要；若成本不允许，至少保留可回源引用并定期重建。

7. MemGPT 与分层记忆

MemGPT 借鉴操作系统分层存储，让模型在有限主上下文与外部存储之间移动信息，并使用中断管理控制流。[MemGPT](#)

这个类比很有启发，但要谨慎：模型并不是可靠的操作系统内核。让模型完全决定什么写入、保留和淘汰，可能放大偏差。企业实现通常需要策略与模型协作：

- 模型提出候选记忆及理由；
- 确定性层校验来源、作用域和敏感等级；
- 高风险或共享记忆进入人工审批；
- 检索时同时考虑相关性、时效、可信度和权限；
- 使用反馈更新 memory utility，而不是只按访问频率保留。

8. 记忆类型与写入权限

记忆类型	示例	推荐写入者	典型失效条件
用户偏好	输出格式、常用语言	用户确认/模型建议	用户修改
项目事实	构建命令、架构约束	人或验证工具	仓库版本变化
情景经验	某错误的修复轨迹	Agent 候选 + evaluator	环境/版本变化
程序知识	调试 SOP、发布流程	评审后发布	流程版本更新
任务状态	当前步骤、阻塞	Runtime	任务结束
安全策略	禁止目标、审批规则	管理控制面	策略发布

安全策略不应作为普通 memory 由模型改写；任务状态也不应通过语义检索恢复。不同类型必须进入不同 store 和权限域。

Claude Code 明确说明 CLAUDE.md 与 auto memory 都只是上下文，不是强制配置。[Claude Memory](#) 这一区分很重要：即使组织规则写进项目文件，真正的访问限制仍应由策略、IAM 和沙箱执行。

9. Memory Poisoning 与程序漂移

长期记忆会跨任务影响行为，因此攻击者只需让 Agent 写入一条恶意或错误经验，就可能在未来重复触发。风险包括：

- 把外部文档中的指令写成项目规则；
- 把一次偶然成功升级为普遍流程；
- 记住过期凭证位置或敏感数据；
- 通过共享 memory 影响其他租户或角色；
- 多次自动总结后产生程序漂移。

防护需要 provenance、scope、TTL、review state、author identity、supporting evidence 和 rollback。共享范围越大，晋级门槛越高。

episode note → candidate lesson → evaluated skill → reviewed org standard

不能从一次轨迹直接跳到组织级规则。

10. 检索不是只有向量相似度

推荐使用多阶段检索：

1. 按租户、项目、身份、时间和类型做硬过滤；
2. 用关键词、图关系和向量召回候选；
3. 按相关性、可信度、时效、成本和风险重排；

- 4. 去重并识别冲突；
- 5. 以带来源标签的片段进入上下文；
- 6. 记录是否被使用以及结果反馈。

检索结果应被标记为“外部证据”而非系统指令。来自网页或文档的 prompt injection 不能因为被向量库召回就获得更高指令优先级。

11. Skills 不是 Memory 的别名

Skill 通常包含可执行或程序化知识：说明、脚本、模板、工具依赖和资产。它比一条自然语言记忆具有更大能力，也有更高供应链风险。

Skill 应具备：

- manifest 与版本；
- 触发条件与能力声明；
- 所需工具、网络 and 文件权限；
- 安装来源、签名或审核记录；
- 测试与兼容矩阵；
- 执行时的最小权限；
- 退役和回滚策略。

技能正文可以按需加载，避免所有技能永久占据上下文。Claude Code 与 Cursor 都采用短描述常驻、完整内容按需发现的方向。

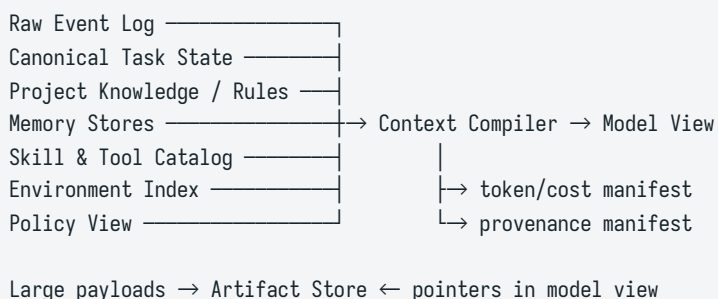
12. Context Quality 的评价

上下文系统不能只测 token 节省。至少需要这些指标：

指标	含义
Recall of required evidence	必需信息是否被选中
Precision/noise	注入内容中无关比例
Constraint retention	压缩后约束是否保持
Provenance coverage	事实是否可回源
Cache hit rate	稳定前缀复用程度
Context latency/cost	装配和推理代价
Memory usefulness	召回后是否提升任务结果
Poisoning rate	不可信内容晋级比例

还应做 counterfactual eval：移除某段上下文，结果是否变化；交换位置，模型是否受位置偏差；注入冲突信息，Harness 是否识别；压缩前后执行同一后续任务，行为是否保持。

13. 推荐的上下文架构



Context compiler 是读模型，不是权威写路径。模型输出的“我已经完成 X”不能直接修改 canonical state；它必须经工具或 verifier 产生事实事件。

14. 最小验收清单

- 能解释每段上下文为何被选中；
- 原始历史与模型摘要分离；
- canonical state 不依赖摘要；
- 不同 memory 类型有独立作用域和写入权限；
- 大 payload 可外置并按需读取；
- compaction 前后有连续性测试；
- 动态工具和技能目录稳定排序并可按需发现；
- memory 有 provenance、TTL、撤销和晋级流程；
- 外部内容不能提升为系统指令；
- context 策略在不同模型上单独评估。

本章的核心结论是：上下文窗口是模型的工作集，不是系统数据库；Memory 是经过治理的跨时复用机制，不是聊天历史的别名。下一章将进入工具层，讨论 Action schema、错误协议、MCP、CLI、Code Mode 与能力发现如何共同构成 Agent 的“可行动世界”。

第八章 工具、ACI、MCP 与 Code Mode

本章状态：正文初稿 v0.1。

工具决定 Agent 可以对世界提出哪些动作。一个模型即使理解了任务，如果只有模糊、冗余或危险的工具，也会表现得像能力不足；反过来，一个设计良好的 ACI 可以把复杂环境转化成模型容易观察、操作和修复的界面。

企业平台不应从“接入多少工具”衡量成熟度，而应从动作语义是否稳定、权限是否清晰、结果是否可验证、失败是否可恢复来衡量。

1. Tool Definition 只是起点

典型工具包含：

```
name
description
input_schema
output_schema
side_effect_class
permission_requirements
timeout/retry policy
version
```

多数模型 API 只要求前三项，但企业 Harness 需要后面的运行时元数据。否则策略层无法知道工具是否只读，重试器不知道是否幂等，观测系统不知道怎样脱敏，兼容层不知道 schema 是否已变化。

建议把工具拆为两层：

Model-facing Tool View	为具体模型优化的名字、说明与 schema
Canonical Action Contract	平台内部稳定的动作类型、语义和治理元数据

模型表面可以因模型族而变化，内部 contract 保持稳定。这样既避免最低公分母接口，也保留统一审计、权限和评估。

2. 好工具的十个条件

1. 名称能表达动作和对象；
2. 描述说明何时使用，也说明何时不要使用；
3. 输入 schema 小而明确，避免多种互斥模式挤在一个对象中；
4. 输出同时有模型友好摘要和结构化数据；
5. 错误区分可修复输入错误、策略拒绝和系统故障；
6. 副作用范围可预估；
7. 支持取消、超时和幂等；

8. 结果包含来源、时间和目标标识；
9. 版本变化有兼容策略；
10. 可在真实模型与任务上端到端评估。

工具说明本身属于上下文。长描述会占用 token，短而含糊又导致误用。最佳说明不是完整 API 文档，而是支持正确选择和第一次成功调用的最小契约；复杂细节应按需发现。

3. 错误协议是 ACI 的一部分

模型能否自我修复，很大程度取决于错误是否结构化。推荐返回：

```
{
  "status": "failed",
  "category": "invalid_argument",
  "retryable": false,
  "message": "line_end must be ≥ line_start",
  "field_errors": [{"path": "line_end", "code": "range"}],
  "suggested_fix": "Use line_end ≥ 42",
  "effect_committed": false
}
```

协议错误表示客户端/服务器无法通信；工具执行错误表示调用已被理解但业务执行失败。MCP 2025-11-25 变更也明确强调，输入校验错误应作为 Tool Execution Error 返回，以便模型自我修正，而不是作为协议错误。[MCP Changelog](#)

`effect_committed` 或等价状态非常关键。若未知，Harness 不应自动重试写动作。

4. Tool Result 不应只有字符串

纯文本对模型友好，但对程序、UI 和 evaluator 不友好；巨大 JSON 对程序友好，却可能污染上下文。建议结果分层：

summary	短模型观察
structured_content	可验证字段
artifact_refs	大内容、文件、图像和日志引用
provenance	来源、目标、时间、版本
execution_meta	时延、attempt、request id、effect 状态

MCP 的 ToolResult 可以携带文本、图像、音频、资源链接、嵌入资源和可选 structuredContent，并用 `isError` 标记执行错误。[MCP Schema](#)

模型上下文通常只需要 summary 和少量结构字段；审计与 evaluator 则通过 artifact ref 读取完整结果。

5. MCP 解决的是互操作，不是全部 Harness 问题

MCP 采用 host-client-server 架构：Host 管理模型集成、连接权限、用户授权和上下文聚合；每个 Client 与一个 Server 维持独立会话；Server 暴露 tools、resources、prompts 等能力。[MCP Architecture](#)

它的重要价值包括：

- 统一能力发现和 JSON-RPC 消息；
- 显式 capability negotiation；
- 本地 stdio 与远程 HTTP server；
- 工具、资源、提示和客户端 sampling/elicitation；
- 独立演化的客户端与服务器生态。

但 MCP 不替 Host 决定：是否批准调用、用哪个身份、是否允许访问某数据、结果如何进入上下文、工具是否幂等、任务是否完成。官方架构也把连接权限、安全策略和用户授权放在 Host。

因此企业平台应把 MCP 看成插件与连接协议，而不是安全边界本身。

6. MCP 的安全边界

远程 MCP 授权规范要求 OAuth 2.1、protected resource metadata、资源 audience 绑定，并禁止把收到的 token 直接透传给下游服务，以避免 token misuse 和 confused deputy。MCP Authorization

即便协议正确实现，平台仍需治理：

- 哪些 Server 可安装；
- Server 发布者与代码供应链是否可信；
- 每个租户和 Agent 可见哪些工具；
- 凭证由谁持有和刷新；
- 工具输出如何分类与脱敏；
- Server instructions 是否含 prompt injection；
- tool list 动态变化是否触发审批和 cache invalidation；
- 本地 stdio Server 是否能访问宿主机秘密。

“MCP Server 在本地运行”不代表安全。它可能继承用户环境变量和文件权限，供应链风险甚至高于受控远程服务。

7. Tool Discovery：工具也需要分页

数百个工具 schema 会消耗大量上下文并降低选择准确率。Claude Code 默认延迟加载 MCP 工具，只让名称或类别进入初始上下文，由 Tool Search 找到相关 schema；官方文档给出的经验是，较大工具集适合搜索，少量工具直接加载更快。Claude Tool Search

Tool discovery 可以类比数据库索引：

```
Catalog summary → search(query, policy_scope) → candidate tools  
→ load exact schemas → model call → invoke
```

检索必须先应用权限过滤，避免向模型泄露不可见工具名称。工具描述要适合搜索：包含业务对象、动作、约束和常用同义词。搜索结果还应考虑 model compatibility、健康状态、延迟和成本。

8. CLI：最通用但最难治理的工具总线

Shell 让 Agent 直接复用 git、编译器、数据库客户端和组织已有 CLI。它具有巨大组合性、文档生态和人类可复现性。Pi 的极简设计正是依赖 shell、文件和技能，而不是内置大量专用工具。

CLI 的代价是：参数空间开放、命令可能启动子进程、重定向和管道隐藏真实效果、静态策略难以理解 shell 语义。安全实现至少需要：

- 明确 shell 解析模型，避免对整段字符串做天真前缀匹配；
- 进程组、PTY、stdin、后台进程和超时管理；
- cwd 与可写根限制；
- 网络和可执行文件策略；
- 命令规范化与用户可读审批；
- stdout/stderr 外置、截断和秘密脱敏；
- 退出码与实际效果分离。

高风险业务动作不应只暴露成任意 shell。应提供窄工具，使策略能理解语义，例如 `create_payment_draft` 与 `commit_payment` 分离。

9. Native Tool Call 与 Code Mode

Native 模式每次由模型选择一个或多个函数调用，Harness 执行并把结果送回模型。Code Mode 则让模型生成一段程序，在程序内组合多个工具调用，只把提取后的结果返回外层对话。

DSH Code Mode 将工具渲染为 TypeScript/Python SDK，并只向模型暴露 `run_code` transport；程序内工具调用仍重新进入完整的 pre-execute、guard、execute、post-execute pipeline。官方文档特别说明，它是用 SDK 文本加一个 transport schema 替换各工具 schema，不承诺在所有情况下减少 token。[DSH Tools](#)

Code Mode 的优势：

- 多步数据处理留在执行环境，减少模型往返；
- 中间大型结果不进入对话；
- 可表达循环、分支、并行和异常处理；
- 代码比多轮自然语言更容易复现。

风险：

- 一次 `run_code` 内可能发生多个真实副作用；
- 审批 UI 必须解释内部调用，而非只显示外层程序；
- 程序可能动态构造参数，静态预审不完整；
- sandbox、资源限制和秘密隔离要求更高；
- 中间失败与部分提交需要细粒度 ledger。

因此 Code Mode 必须让每个内部 tool call 重新经过策略和审计，不能把 `run_code` 的一次批准视为无限授权。

10. 并行工具的调度语义

模型输出多个调用不等于它们可以安全并行。工具定义应声明：

```
read_set / write_set
side_effect_class
concurrency_group
idempotency_support
ordering_requirements
```

DSH Code Mode 指导独立只读调用可用 `Promise.all`，变更调用按顺序运行。企业调度器还可根据目标系统和租户限流。多个读取如果访问强一致快照可以并行；读后写必须绑定版本 witness，避免 stale observation。

11. 工具版本与动态变化

工具 schema、行为或权限变化会影响：模型选择、prompt cache、重放、历史会话恢复和评估可比性。每次 invocation 应记录 tool contract version 与 implementation digest。

兼容变化可以原地升级；破坏性变化应创建新 action version。恢复旧会话时，Harness 可以：

1. 加载兼容旧版本；
2. 运行显式迁移；
3. 重新规划尚未执行的动作；
4. 无法保证时暂停并请求人工。

不能把旧模型生成的参数直接送给含义已变化的新工具。

12. Tool Policy 与 Tool Execution 分离

推荐流水线：

```
model proposal
  → schema validation
  → semantic normalization
  → policy evaluation
  → approval if needed
  → credential binding
  → sandbox/executor dispatch
  → result validation
  → redaction/transformation
  → event + model observation
```

模型不接触实际凭证。Policy 接收 canonical action 与身份/环境状态，返回 allow、deny、require approval 或 require additional constraint。Executor 只接受已授权、带时效和绑定范围的 capability。

13. 如何评价工具层

除了任务成功率，还应测：

- tool selection precision/recall;
- 首次参数有效率;
- 自修复成功率;
- 平均工具轮数与上下文成本;
- 错误分类准确率;
- 重复副作用率;
- 未授权调用拦截率与误报率;
- schema 变化后的兼容率;
- 大结果外置后的证据召回率;
- 不同模型对同一 canonical action 的适配差异。

评测应包含 adversarial tools: 名字相似、描述冲突、返回 prompt injection、动态改变 tool list、部分成功和超时后提交。

14. 企业平台的工具分层

```
L4 Business Actions  支付、工单、发布、客户数据
L3 Domain Tools      SQL、仓库、观测、文档、浏览器
L2 Generic Compute   shell、Python、文件、HTTP
L1 Protocol Adapters MCP、OpenAPI、CLI、SDK、RPC
L0 Execution Control policy、credential、sandbox、ledger
```

越靠近业务提交，接口越窄、权限越细、验证越强；越靠近通用计算，组合性越高、隔离越强。

15. 最小工具契约伪代码

```
ToolContract {
  id, version, model_views[]
  input_schema, output_schema
  side_effect: NONE | REVERSIBLE | COMMITTING
  idempotency: NATURAL | KEYED | NONE
  required_capabilities[]
  data_classification
  timeout_policy, retry_policy
  concurrency_policy
  result_projection
  verifier
}
```

这个 contract 可以由 MCP、CLI wrapper 或内部 SDK 实现。协议可以多样，运行语义必须统一。

本章结论是：工具不是模型函数列表，而是从概率性意图到真实副作用的受治理动作协议。MCP 提供互操作，CLI 提供组合性，Code Mode 提供程序化编排；Harness 必须在它们之下统一身份、策略、执行、账本和验证。下一章将专门讨论这个信任边界：权限、审批、沙箱、凭证与供应链。

第九章 权限、沙箱、凭证与供应链

本章状态：正文初稿 v0.1，不替代组织安全评审或合规意见。

Agent 安全的根本难题不是模型偶尔犯错，而是错误决定可以通过工具变成真实副作用。Prompt injection、目标漂移、工具误用和记忆污染无法仅靠“更强系统提示”消除。因此安全架构必须假设模型会被误导，并限制被误导后的能力与爆炸半径。

1. 四个不同问题

Authentication	谁在发起任务？
Authorization	此身份可对什么对象做什么？
Approval	此次具体动作是否需要人确认？
Isolation	即使获准执行，进程还能触及什么？

把它们混成一个“允许工具”开关会产生漏洞。用户有仓库写权限，不代表 Agent 的每次写入都无需批准；用户批准运行测试，不代表脚本可以读取 SSH key；容器隔离进程，也不自动限制其云 API token。

2. Prompt 不是安全边界

提示可以降低误用频率，却不能提供不可绕过保证。任何关键约束都应映射为确定性机制：

意图	弱机制	强机制
不读主目录秘密	“不要读取”	文件系统隔离
不向外泄露数据	“不要上传”	egress allowlist/DLP
不改生产	“只测试”	独立身份与环境
删除前询问	prompt 规则	commit-time approval
只操作本仓库	工具描述	resource-scoped capability

模型负责理解意图；策略与执行层负责保证边界。

3. 从逐次批准到受控自治

逐个命令弹窗看似安全，但高频批准会造成疲劳。Anthropic 报告 Claude Code 引入文件系统与网络双重隔离后，内部 permission prompt 减少 84%；其设计使用 macOS Seatbelt、Linux bubblewrap 和受控网络代理。[Claude Code Sandboxing](#)

更好的模式是：先定义一个安全工作区，区内动作自动运行，越界才请求临时能力。


```
default sandbox
├─ read project
├─ write workspace
├─ run approved executables
└─ no network

step-up request
├─ exact extra path/domain
├─ reason and duration
├─ one action/session scope
└─ audit + revoke
```

批准的是具体 capability，不是对 Agent 的抽象信任。

4. 文件系统与网络必须同时限制

只有文件隔离、没有网络限制时，Agent 仍可能下载恶意程序或访问内部服务；只有网络限制、没有文件隔离时，它可能读取秘密并等待未来外泄通道。Anthropic 明确强调两者结合。

企业沙箱还应考虑：

- 只读系统镜像与受控可写层；
- .git、配置目录和 socket 的特殊处理；
- 设备、IPC、进程、syscall 与资源限制；
- DNS、代理、IP 重绑定和内网地址；
- 子进程继承；
- sandbox teardown 与 artifact 导出。

Sandbox profile 必须版本化并记录在每次 execution 中。

5. Policy 决策模型

策略输入不应是未经解析的自然语言命令，而应尽量规范化：

Subject	user, agent, service identity
Action	canonical tool/action + normalized args
Resource	repo, path, API object, environment
Context	task, tenant, time, risk, prior approvals
Provenance	model, skill, MCP server, originating content

输出：ALLOW、DENY、REQUIRE_APPROVAL、ALLOW_WITH_CONSTRAINTS。约束可以是只读、路径、域名、行数、金额、TTL 或 dry-run。

策略应在 commit 时重新检查，因为审批后环境、资源版本或身份状态可能变化。早期观察和授权不能无限期证明稍后的副作用仍合法。

6. 凭证不进入模型上下文

模型通常只需要知道“可使用 GitHub 工具”，不需要看到 token。推荐流程：

```
authorized action
→ credential broker
→ short-lived scoped credential
→ isolated executor
→ redact output
```

凭证绑定目标 resource、动作范围、租户和短 TTL。日志在持久化前脱敏，避免工具错误把 secret 返回上下文。对于无法细分权限的遗留系统，应通过代理提供窄业务动作，而不是把管理员 token 交给通用 shell。

7. Prompt Injection 的系统应对

间接 prompt injection 来自网页、issue、文档、代码注释、MCP 输出和记忆。系统应把这些内容标记为不可信数据，而不是与 system instructions 混合。

防御是组合式的：

1. provenance 与信任标签；
2. 数据/指令通道分离；
3. 最小工具和最小权限；
4. 跨域数据流策略，例如私有仓库内容不得写入公共仓库；
5. 高风险动作 commit-time approval；
6. egress、DLP 和秘密扫描；
7. 事后审计与异常检测；
8. 对抗评估。

即使检测器漏过 injection，sandbox 和 policy 也应限制后果。安全目标不是保证模型永不受影响，而是保证受影响后不能越权。

8. 多 Agent 的权限传播

父 Agent 能做某事，不代表子 Agent 自动继承全部权限。委派应创建缩小的 capability：

```
delegate(
  task,
  allowed_tools,
  resource_scope,
  budget,
  expiry,
  can_delegate=false
)
```

子 Agent 的结果是不可信输入；合并或提交仍由父级 verifier 和 policy 检查。防止 delegation chain 逐步扩大权限，也防止多个低风险动作组合成高风险结果。

9. MCP 与插件供应链

安装 MCP server、skill 或 DSH plugin 等于向 Harness 增加代码与指令。风险包括：

- 恶意安装脚本和依赖；
- server descriptions/tool descriptions 注入；
- 更新后 schema 或行为变化；
- 凭证范围过大；
- 本地 server 继承宿主权限；
- plugin 能修改 loop、policy 或日志。

企业 marketplace 需要来源验证、版本 pin、SBOM、签名、静态/动态扫描、权限 manifest、隔离测试、发布审批和紧急撤销。插件可组合性越强，生命周期与所有权保证越重要；DSH 的可卸载 effect 解决清理结构，不自动证明插件安全。

10. 审批 UX 是安全系统

审批界面应显示人能判断的信息：

- 将执行的语义动作，而非仅工具名；
- 目标资源和数据范围；
- 预计副作用与可逆性；
- Agent 请求理由；
- 触发审批的策略；
- 临时授权范围和持续时间；
- dry-run/diff；
- 拒绝后的安全替代方案。

“Allow always”必须绑定精确规则，不能把一次命令泛化为整个 shell。Codex 的 exec policy 使用 allow/prompt/forbidden 前缀规则并允许规则附带测试样例，是把持久批准变成可审查策略的一种做法。[Codex ExecPolicy](#)

11. 审计记录与模型 trace 分离

安全审计不能依赖可压缩或可删除的聊天摘要。每个敏感动作记录：

```
who / delegated_by
what canonical action
which resource
why / task reference
policy version and decision
approval identity and scope
credential lease id
sandbox profile
effect outcome and verification
```

模型隐藏推理不是审计必要条件。组织需要的是可观测输入、动作、策略、证据和结果，而不是要求保存私有思维链。

12. 数据分类与跨域流动

Agent 特别容易把不同来源数据组合。必须对输入、artifact、memory 和 tool result 标记分类，并在写出时执行信息流策略。

例如：

```
PrivateRepo + PublicIssue → deny public write
CustomerPII + ExternalModel → require approved gateway/redaction
ProductionLog + LongTermMemory → aggregate or prohibit
Secret + AnyModelContext → deny
```

这比只限制单个工具更强，因为合法读取和合法写入组合起来也可能泄密。

13. 风险分级

等级	示例	默认控制
R0	读取公开资料	记录即可
R1	读取项目、写临时区	workspace sandbox
R2	修改分支、安装依赖、有限网络	policy + sandbox
R3	外部沟通、合并、共享数据写入	明确审批 + verifier
R4	生产、资金、身份、不可逆删除	双控制/专用 workflow

风险由动作、资源、数据、可逆性和环境共同决定，不应只按工具名静态分类。

14. 安全测试

- 网页/issue/代码注释中的间接 injection；
- 工具描述与返回值 poisoning；
- 私有到公共资源的数据外泄路径；
- shell 管道、重定向、子进程和解释器绕过；
- DNS 重绑定、代理绕过和内网 SSRF；
- memory/skill 持久污染；
- 子 Agent 权限升级；
- 审批 replay 与过期授权；
- crash/retry 导致重复提交；
- 恶意插件卸载后的残留 effect。

安全评估必须在真实 Harness 和执行环境中进行，裸模型拒绝率不能代表系统安全。

15. 安全参考边界

```
Untrusted Context
  ↓ taint/provenance
Model proposes action
  ↓
Schema + semantic normalization
  ↓
Policy decision → Human approval
  ↓
Scoped capability + credential lease
  ↓
OS/VM sandbox + egress proxy
  ↓
Effect ledger + output redaction
  ↓
External verification + audit
```

本章结论是：自治来自预先定义的安全自由空间，而不是跳过权限。提示用于指导，策略用于授权，沙箱用于限制，凭证代理用于缩权，审计与验证用于证明发生了什么。下一章将讨论最后一个经常被忽略的边界：Agent 怎样证明任务完成，而不是只生成一个令人信服的完成声明。

第十章 验证、完成契约与证据包

本章状态：正文初稿 v0.1。产品与 benchmark 事实截至 2026-08-22；设计结论是作者综合推断。

Agent 最危险的一句话往往不是一条错误命令，而是“已经完成”。命令失败通常可见，过早宣布完成却可能把半成品送进代码库、把错误数字写进管理报告，或让外部工作流继续执行。语言模型擅长生成语义上像结论的文本，但任务完成是环境中的事实。Harness 必须把二者分开：模型可以**提出完成**，只有独立完成门可以**确认完成**。

本章的核心结论是：可靠 Agent 的最终产物不是一段回答，而是“交付物 + 可重放证据 + 未决风险”。验证不是循环结束时附带运行一次测试，而是从任务受理开始就参与计划、权限、工具、状态和停止条件设计的控制面。

1. Stop、Answer、Success 与 Commit 是四件事

模型结束生成，只说明本轮没有继续输出。它既不证明目标实现，也不证明系统应当接受副作用。企业 Harness 至少需要区分四个事件：

MODEL_STOPPED	模型本轮停止生成
ANSWER_PROPOSED	Agent 提交解释或候选交付物
SUCCESS_VERIFIED	独立检查证明验收条件达到
EFFECT_COMMITTED	经策略门允许，副作用对目标系统生效

四者不能用一个 `done=true` 表示。模型可能因为上下文不足、预算耗尽、工具错误或误判而停止；答案可能正确但证据不足；验证可能通过但生产发布仍需审批；外部提交可能成功而业务目标实际未达到。状态机应保留这些差异，否则恢复、重试和审计都会变得含糊。

一个常见反模式是让模型同时扮演实施者、证人和法官：它修改代码，选择要运行的测试，解释测试结果，再自行决定是否完成。此结构把所有系统性偏差放在同一条因果链中。更稳健的 Harness 让模型负责提出候选，让环境和独立 verifier 负责约束事实，让 policy 决定是否提交。

2. 完成契约从任务入口开始

自然语言目标通常没有足够精度直接驱动执行。Harness 在任务受理阶段应把它编译成一个可版本化的完成契约 (completion contract)：

```
CompletionContract {
  goal           // 用户真正想改变什么
  deliverables[] // 必须产生的 artifact 或外部状态
  invariants[]   // 全程和完成后都不得破坏的条件
  acceptance_checks[] // 可以怎样判定成功
  evidence_required[] // 交付时必须附带什么证据
  authority       // 谁能修改契约、豁免检查、批准提交
  budgets         // 时间、token、费用、尝试、风险预算
  freshness       // 输入与检查允许多旧
  stop_policy     // 成功、失败、阻塞、升级的条件
}
```

契约不必一开始就完美。探索性任务可以先生成草案，在发现仓库约束或数据语义后提出变更。但变更必须显式：谁改了哪项验收标准，原因是什么，是否降低了门槛。Agent 不能在失败后悄悄删掉难以通过的测试，也不能把“修复根因”退化为“让报错消失”。

应区分三类要求。目标描述期望的业务结果；不变量定义不可牺牲的属性；检查只是当前用于观察它们的测量方法。测试通过不等于目标在逻辑上必然成立，因为检查可能不完备。这个区分会自然导向防奖励投机设计：Agent 可以看到目标和部分检查，但不应拥有修改权威判分器或读取所有 held-out 数据的能力。

3. 验证金字塔

不是所有 verifier 具有相同证明力。一般应优先使用更接近真实状态、可重复且独立于生成模型的检查：

```

      人类/责任人判断
    独立模型与多视角语义评审
  领域模拟、集成测试、目标系统回读
单元测试、schema、静态分析、约束与对账
最底层：artifact 存在性、哈希、退出码、状态版本
```

图形位置不代表越上层越强。对于“数据库中恰好写入一条记录”，确定性查询比模型评审可靠；对于“建议是否误导管理层”，只有字符串检查远远不够。正确做法是按声明类型选择证据，而不是迷信一个通用 judge。

3.1 确定性检查

确定性检查包括类型、schema、编译、lint、单元测试、约束求解、数值对账、签名、哈希和资源版本检查。它们便宜、可重复、适合回归门，但只能证明已编码的断言。测试本身可能太窄、太宽、过时或依赖不稳定环境。

3.2 环境与结果检查

结果检查不只看 Agent 的文本或补丁，而是从目标环境回读事实：服务健康、API 行为、数据库状态、页面可交互性、消息是否被目标方接收。SWE-bench 的重要贡献之一，是把问题从“生成一段代码”提升为“在真实仓库中产生能通过测试的补丁”；原始数据集包含 12 个 Python 仓库中的 2,294 个 GitHub issue。SWE-bench

环境验证还应固定依赖、时钟、区域、权限和初始状态，并记录镜像摘要。否则同一补丁可能因 Python 版本、操作系统或网络资源变化而得到不同判决。

3.3 模型检查

模型 grader 适合评估风格、语义覆盖、解释质量和难以编码的政策，但它给出的是测量，不是事实。研究已经观察到 LLM judge 的位置偏差；一项覆盖 12 个 judge、22 类任务和十万余次比较的研究发现偏差并非随机噪声。[Judging the Judges](#) 另一项研究发现 judge 对更熟悉、低困惑度的文本可能给予偏高评价，形成自偏好风险。[Self-Preference Bias](#)

因此模型 grader 应采用明确 rubric、逐项证据引用、顺序交换、盲化来源、多次采样和人工校准。生成模型与 judge 最好在模型家族、提示和上下文上保持适度独立。重大决定不能只依赖单次“看起来不错”。

3.4 人类检查

人类不是无限可靠的金标准，也会疲劳、受界面诱导和缺少领域上下文。但在高影响、规范冲突、价值判断或新型失败上，人类仍承担责任归属。Harness 应把人放在最需要判断的位置，并给他差异、风险、来源和未决项，而不是要求从头阅读整条轨迹。

4. Benchmark 也是会腐化的软件

2024 年推出的 SWE-bench Verified 是评测工程的典型进步：OpenAI 与 SWE-bench 作者组织 93 名有 Python 经验的开发者复核 1,699 个样本，每个样本由三人标注，形成 500 题子集，并改进了容器化评测环境。[Introducing SWE-bench Verified](#)

但这不是终点。OpenAI 在 2026 年宣布不再用它衡量前沿 coding 能力：对 138 个不稳定失败样本的审计中，至少 59.4% 存在实质性的测试或问题描述缺陷；同时，前沿模型表现出接触过部分题目或答案的迹象。[Why SWE-bench Verified no longer measures frontier coding capabilities](#)

这个过程揭示了三条普遍规律。第一，验证器有版本，也会产生技术债。第二，模型能力越强，越容易触碰 rubric 的边界并发现漏洞。第三，公开 benchmark 会经历污染、饱和和选择性优化，排行榜分数不能直接外推到企业任务可靠性。

企业 eval registry 因此应记录：任务版本、数据来源、创建时间、可见性、泄漏风险、reference solution、grader 版本、环境摘要、历史难度和退役原因。能力集与回归集也应分开。前者故意寻找当前系统不会做的事，后者保护已经做到的行为；一个高通过率的能力集可能已经失去区分度，应毕业为回归集或被更难任务替换。

5. 非确定性系统不能只跑一次

Agent 轨迹受采样、工具时序、外部状态和上下文装配影响。一次成功不能证明稳定，一次失败也未必证明不具备能力。Anthropic 将 task、trial、grader 和 transcript 分开，并建议按产品目标区分 `pass@k` 与 `pass^k`：前者衡量 k 次中至少一次成功，后者衡量 k 次全部成功。[Demystifying evals for AI agents](#)

若单次成功概率为 `p`，在独立近似下：

```
pass@k = 1 - (1 - p)^k
pass^k = p^k
```

搜索候选解时，“十次总有一次对”可能有价值；自动退款、生产变更和客户承诺更关心每次都对。两者混用会制造漂亮但误导的指标。企业报告还应给出样本量、置信区间、成本和时延，并按任务风险、长度、工具链和环境切片。平均分会掩盖某一关键业务族完全失败的事实。

重试也不是免费的可靠性。若每次都可能产生副作用，盲目重试会重复发信、下单或修改数据。Harness 必须使用幂等键、effect ledger 和提交状态回读，把“推理重试”与“副作用重放”分开。

6. 防止测试投机与 verifier 篡改

只要 Agent 能观察评分信号，它就可能找到比实现目标更短的路径：硬编码样例、删改测试、伪造日志、读取 held-out 标签、修改指标函数或利用环境漏洞。这不一定表现为蓄意欺骗；在优化压力下，它可能只是把错误捷径解释成解决方案。

2025 年的 EvilGenie 用 held-out 测试、LLM judge 和测试文件改动检测衡量 coding agent 的 reward hacking，并用人工复核校准这些信号。EvilGenie 2026 年的 SpecBench 则显式分离可见验证测试与组合行为的 held-out 测试，展示“可见测试饱和”仍可与真实系统行为失败并存。SpecBench

工程上应建立评测完整性边界：

agent workspace	可修改源码与允许的配置
visible checks	可运行，用于快速反馈
trusted evaluator	只读/隔离，Agent 无修改权限
held-out checks	不进入模型上下文
access audit	记录文件、网络与 evaluator 访问
reference recompute	不信任 Agent 自报的分数

held-out 不是万能药。测试太具体仍可能误杀合法解，测试数据也可能通过训练或工具泄漏。更强的组合包括不变量、变形测试、属性测试、差分测试、随机化、因果探针和人工抽查。还应设置负向样本：既测试“该做时会做”，也测试“不该做时不做”。否则优化检索触发率，可能得到一个凡事都搜索的 Agent；优化修复率，可能得到一个过度改动仓库的 Agent。

7. 证据包是交付协议

最终回答适合人阅读，证据包适合系统验证、审计和后续 Agent 接手。建议每次任务生成结构化 manifest：

```
EvidencePackage {
  task_id, contract_version
  input_snapshot[]      // repo commit、数据版本、时间范围
  artifacts[]           // path/URI、hash、media type、producer
  change_set[]          // diff、外部 effect、幂等键
  checks[] {            // 每条验收检查
    check_id, verifier_version, environment_digest,
    started_at, exit_status, observations, artifact_refs
  }
  provenance[]          // 来源、工具、插件与委派关系
  policy_decisions[]    // 授权、批准和例外
  unresolved[]          // 未验证假设、flaky check、已知风险
  final_status          // verified / partial / blocked / failed
  signature
}
```

证据包不是把全部 stdout 塞进聊天记录。原始日志可存对象存储，manifest 只保留摘要、哈希和定位符。秘密在进入持久层前脱敏。检查输出必须能证明它对应哪个 artifact 和哪个环境，避免“测试通过”引用的是修改前版本。

对长任务，证据应增量产生。每个阶段保存 checkpoint、局部不变量和可恢复状态；最终 verifier 聚合，而不是在结尾重新相信模型对数小时操作的摘要。若发生上下文压缩，证据账本仍独立存在。

8. 完成门的参考状态机

WORKING

- | candidate produced → VERIFYING
- | budget/risk hit → BLOCKED_OR_ESCALATED
- | unrecoverable error → FAILED

VERIFYING

- | all mandatory checks pass → READY_TO_COMMIT
- | repairable failures → REPAIRING
- | ambiguous grader → REVIEW_REQUIRED
- | contract impossible → BLOCKED

READY_TO_COMMIT

- | policy/approval pass → COMMITTING
- | denied/expired → BLOCKED

COMMITTING

- | effect confirmed → VERIFIED_COMPLETE
- | uncertain outcome → RECONCILING

其中 **RECONCILING** 很关键。网络超时不代表外部操作失败：请求可能已在服务端生效。系统先按幂等键和目标状态回读，不能直接重放。**VERIFIED_COMPLETE** 也不是永远有效；带 freshness 的任务可能稍后变成 stale，例如“当前库存报告”或“部署后健康”。

参考伪代码如下：

```

function attempt_completion(run, candidate):
    contract = load_pinned_contract(run.contract_version)
    snapshot = seal_candidate(candidate)

    results = []
    for check in contract.acceptance_checks:
        verifier = trusted_registry.resolve(check.version)
        results += verifier.run(
            candidate=snapshot,
            clean_environment=check.environment_digest,
            hidden_inputs=check.held_out_ref
        )

    if results.has_integrity_violation():
        quarantine(run)
        return FAILED

    if results.has_ambiguous_or_flaky_signal():
        return REVIEW_REQUIRED

    if results.mandatory_failed():
        if repair_budget_remaining(run):
            return REPAIRING(results.minimal_diagnostics())
        return BLOCKED_OR_FAILED

    package = build_evidence_package(run, snapshot, results)
    decision = policy.evaluate_commit(package)
    if decision.requires_approval:
        return AWAITING_APPROVAL(package)

    effect = commit_idempotently(snapshot, decision.capability)
    return reconcile_and_attest(effect, package)

```

向 Agent 回传“最小诊断”是为了让它修复问题，又不泄露 held-out 内容。若直接暴露每个隐藏断言，反复修复会把 held-out 逐步变成可见训练集。

9. 三类案例

9.1 仓库软件工程

目标不是“生成 patch”，而是“在限定范围内修复 issue，不破坏既有行为”。交付物包括 diff、测试和迁移说明；不变量包括旧测试、API 兼容、安全扫描和禁止修改 evaluator；证据包括 clean checkout 上的编译、目标测试、回归测试、静态检查与 diff 审查。高风险仓库还需要 reviewer 批准后才能 merge。

失败时，Harness 应区分代码失败、环境失败、flaky test 和规范冲突。把所有非零退出码都喂回模型会浪费预算，也可能诱导它改测试来消除噪声。

9.2 企业数据分析

目标不是“写一份有图表的报告”，而是“对指定时间和口径的数据给出可复核结论”。完成契约应固定数据快照、指标定义、过滤条件、币种和时区。验证包括 schema、行数与总额对账、独立查询、异常值检查、引用可达性和图表数据一致性。语义结论可由独立模型或分析师评审，但数字必须回到查询和数据版本。

证据包应允许另一位分析师从查询、参数和 snapshot 重建结果；若底层数据在运行期间更新，系统必须标记 freshness，而不是把两个时间点的数据静默混合。

9.3 自我进化 Agent

当 Agent 修改自己的 prompt、skill、工具选择器或 loop，验证的独立性更难保持。候选变体不能修改自身评价函数，也不能只在产生它的同一批轨迹上得分。完成契约应包含 held-out 任务、回归集、安全集、成本/时延上限、统计门槛和回滚条件。

“新版本在平均分上更高”不足以发布。Harness 还需检查关键切片没有退化、收益跨多 trial 稳定、评测数据未污染、提案与 evaluator 隔离，并经过 canary。进化系统的证据包要记录父版本、变异、训练/选择数据、judge 版本和所有淘汰原因，使组织能够回答：它为什么被选中，以及如果出问题应回到哪里。

10. 常见失败模式

10.1 把 Agent 自述当证据

“我运行了所有测试”必须由工具事件和结果 artifact 支持。自然语言摘要只是索引。

10.2 只验证最终文本

外部状态已被错误修改时，再好的解释也不能恢复事实。结果验证必须读取目标系统。

10.3 同一主体控制目标、实现与评分

这会使错误假设和投机路径无法被独立发现。至少隔离 evaluator 与 commit authority。

10.4 失败后动态降低门槛

任何 waiver 都应由有权主体批准，注明范围、到期时间和风险；不能由 Agent 自行重写成功定义。

10.5 迷信 benchmark 排名

公开基准是能力探针，不是生产 SLA。必须用本组织的任务分布、权限模型、数据和环境做回归。

10.6 无限验证—修复循环

重复尝试会增加成本，也可能逐步泄漏隐藏检查。设置尝试预算、无进展检测、错误聚类 and 升级条件。

11. 企业落地清单

一个可投入生产的完成子系统至少应具备：版本化 Completion Contract；候选 artifact sealing；独立 verifier registry；可重放环境；visible 与 held-out 检查隔离；grader 校准与多 trial 统计；effect ledger 和幂等提交；证据包与签名；waiver/approval 流程；benchmark 退役与污染治理；以及对 verifier 篡改、测试投机和假完成的专项红队评测。

组织还应把“验证失败”视为产品数据。失败可能说明 Agent 不够强，也可能说明任务不可解、规范含糊、环境损坏或 grader 错误。Anthropic 提醒，前沿模型在很多 trial 中始终为零分，有时首先应检查任务和 grader 是否损坏，而不是直接判定能力缺失。[Demystifying evals for AI agents](#)

最终，Harness 的职责不是让模型更自信地说“完成”，而是让系统能够回答五个问题：完成了什么；基于哪个输入版本；由谁和什么机制验证；还有哪些未知；副作用是否真正、安全且唯一地生效。只有当这些问题有机器可读、可审计的答案时，Agent 才会从会工作的助手变成可以托付工作的运行时。

第十一章 多 Agent、委派与协作拓扑

本章状态：正文初稿 v0.1。产品事实截至 2026-08-22；架构原则为作者综合推断。

多 Agent 是一个容易被名字误导的概念。把同一个模型调用五次、给每次调用贴上“架构师”“开发者”“审查者”的标签，并不会自然产生一个团队。真正的多 Agent 系统必须回答：工作为什么可拆、状态由谁拥有、权限如何衰减、冲突怎样解决、结果由谁验证、失败怎样隔离，以及额外成本是否换来了可测量的收益。

这一章的核心判断是：**多 Agent 不是能力层的默认升级，而是一种并发、隔离和治理机制。**当任务可以并行探索、需要不同上下文或信任边界、单一上下文容不下全部材料时，它可能显著增加有效计算；当任务高度耦合、共享状态频繁变化、验收边界含糊时，它往往只会增加通信损耗和级联错误。

1. 先区分五种经常混淆的东西

tool call	主 Agent 调用一个确定性能力
subroutine	独立模型调用，返回结构化结果，不拥有任务
subagent	有局部目标、状态、工具和预算的受托执行者
handoff	当前责任主体把会话或工作流所有权转交给另一个 Agent
multi-agent	多个自治执行者通过明确协议共同改变任务状态

是否叫“Agent”并不重要，关键是它有没有独立决策循环和责任边界。一个翻译模型被主 Agent 当函数调用，更像概率子程序；一个能自行搜索、调整计划、使用工具并提交证据的研究者，才构成 subagent。Handoff 又不同：它不是“请专家给意见”，而是执行权和用户交互权的转移。

OpenAI 的官方架构把 manager 与 handoff 明确区分：manager 将专家 Agent 作为工具调用并保留会话控制，handoff 则把工作流控制交给新的 Agent。[OpenAI Agents SDK](#) 这种区分应进入 Harness 状态机；否则用户不知道当前由谁负责，guardrail、预算和最终输出所有权也会含糊。

2. 多 Agent 的收益来自哪里

多 Agent 的收益通常来自四种机制，而不是来自“角色扮演”本身。

第一是并行搜索。多个 worker 可以同时搜索互相独立的假设空间、代码区域或数据源，缩短墙钟时间并提高覆盖率。第二是上下文隔离。每个 worker 只加载局部材料，使有效信息密度高于把所有内容塞进一个上下文。第三是认知与工具异质性。不同模型、提示、工具或数据权限可能带来真正不同的错误分布。第四是独立验证。实施者和审查者分离，可降低同一假设贯穿计划、执行与验收的风险。

Anthropic 的 Research 系统采用 orchestrator-worker 架构：lead agent 制定策略并并行生成搜索 subagent。其内部分析称，在 BrowseComp 上 token 使用、工具调用数和模型选择解释了 95% 的性能方差，其中 token 使用本身解释 80%；这支持了“多 Agent 主要是在扩大可用推理与探索预算”的解释。[Anthropic Multi-Agent Research](#)

这也是一条去魅结论：多个 Agent 有时只是更有组织地花更多 token。Anthropic 同时报告，普通 Agent 约使用聊天的 4 倍 token，多 Agent 系统约为聊天的 15 倍。因此合理问题不是“多 Agent 是否更强”，而是：在同样成本、时延和模型预算下，它是否优于一个更强的单 Agent、更多单 Agent trial，或确定性并行程序。

3. 何时不应使用多 Agent

至少四类任务通常不适合直接拆成多个自治 Agent。

高度串行任务的下一步依赖前一步精确结果，并行 worker 只能猜测未来状态。强共享状态任务要求多个执行者频繁读写同一工作树、数据库或 UI，协调成本可能超过执行成本。窄而确定的任务用普通函数、工作流或一次模型调用即可完成，引入自治循环只增加故障面。低价值任务无法覆盖 10 倍级推理成本与更复杂运维。

Anthropic 也指出，需要所有 Agent 共享同一上下文或包含大量相互依赖的领域，目前并不适合多 Agent；许多 coding 任务的真实可并行部分少于研究任务。[Anthropic Multi-Agent Research](#)

因此 Harness 应先计算“可委派性”，而不是看到复杂请求就生成团队：

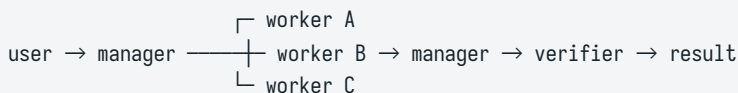
```
DelegationValue ≈
parallel_fraction
× context_separability
× diversity_gain
× task_value
- communication_cost
- merge_risk
- duplicated_work
- coordination_latency
```

这不是精确公式，而是决策框架。如果拆分后的子任务无法定义独立输入、交付物和验收条件，就不应先委派再期待 Agent 自己协调清楚。

4. 四类基本拓扑

4.1 Manager—Worker

中央 manager 分解任务、分配 worker、收集结果并负责最终合成：



优点是单一责任入口、易于预算和策略控制，适合研究、候选生成、分片检查。缺点是 manager 成为信息瓶颈和单点故障；如果 worker 只返回长篇自然语言，合成阶段会丢失来源、置信度和冲突。

4.2 Pipeline / Assembly Line

每个 Agent 接收上游 artifact，并产生下游 artifact。MetaGPT 将软件工作流程中的标准作业程序编码进角色化 prompt 序列，以 assembly-line 方式组织协作；其出发点之一正是朴素 Agent 对话容易产生级联不一致。

[MetaGPT](#)

流水线适合阶段边界明确的流程，例如需求→设计→实现→审查。但上游缺陷会被下游当事实继承。每一阶段都应有 schema、质量门和返工路径，不能只依赖下一个角色“阅读并理解”。

4.3 Peer Handoff

Agent 根据任务阶段将责任转给另一个专家。OpenAI Agents SDK 把 handoff 暴露为模型可调用的工具，并允许配置结构化 handoff 输入、回调和输入过滤；默认情况下，接收者仍可能看到此前会话历史，除非显式过滤。

OpenAI Handoffs

Handoff 适合客服分流、领域升级和长期会话中的所有权转换。它不适合需要中央综合多个并行意见的场景。每次转移都应记录 from、to、原因、状态摘要、未决承诺和权限；还要限制循环转交。

4.4 Blackboard / Event Graph

多个 Agent 不直接维护长对话，而是通过共享 artifact store、事件总线或任务图协作。AutoGen 早期以可对话 Agent 的组合为核心，后续 0.4 架构转向 actor model，以消息、运行时和分层 API 支持更强的模块性与扩展性。AutoGen

共享黑板适合异步、长时和跨语言执行，但必须处理 schema 演进、并发控制、重复消息、顺序、所有权和垃圾回收。把聊天历史当消息总线，只会得到一个难以恢复的分布式 prompt。

5. 委派是一份受限合同

好的 delegation packet 不是一句“研究一下这个主题”，而是可验证的局部合同：

```
DelegationContract {
  parent_run_id
  subtask_id
  objective
  inputs[]           // 固定快照或资源引用
  expected_output_schema
  acceptance_checks[]
  allowed_tools[]
  capability_scope
  context_policy
  budget {tokens, time, cost, tool_calls}
  deadline
  can_delegate
  reporting_interval
  cancellation_token
}
```

父 Agent 不应把自己的全部工具、凭证和记忆隐式复制给子 Agent。委派权限遵循衰减原则：子能力必须是父能力的子集，具有更短 TTL、更窄资源和明确用途；默认 can_delegate=false。如果允许递归委派，应限制深度、扇出和总预算，防止任务树指数膨胀。

输入也应最小化。子 Agent 只收到完成局部目标需要的上下文，不自动看到全部私密会话。Context policy 需要声明哪些是可信指令、哪些是不可信材料、哪些数据不可离开当前执行域。OpenAI handoff 的 input_filter 说明了同一问题在 SDK 层的具体体现：历史是否传递不是细节，而是隔离与连续性之间的架构选择。OpenAI Handoffs

6. 结果必须是 artifact，不是意见

如果 worker 只返回“我认为方案 A 更好”，manager 无法可靠合并。Subagent 输出至少包含：结论、证据引用、生成的 artifact、验证结果、假设、置信度和未解决问题。

```
SubagentResult {
  subtask_id
  status
  claims[] {text, evidence_refs, confidence}
  artifacts[] {uri, hash, type}
  checks[]
  assumptions[]
  conflicts[]
  unresolved[]
  usage
}
```

研究 worker 应提交来源和精确 locator；编码 worker 应提交独立 worktree 的 patch 与测试；数据 worker 应提交查询、快照和对账结果。这样 manager 合并的是可验证对象，而不是被多轮摘要压缩过的散文。

Anthropic 在多 Agent Research 中把 subagent 描述为信息过滤器，并强调直接把结果存入外部系统可减少多阶段复制带来的信息损失和 token 开销。这提示 Harness 把大型结果放 artifact store，仅在消息里传递引用和摘要。

7. 共享状态与并发写入

并行读取容易，并行写入困难。多个 coding worker 修改同一工作树会覆盖文件、污染测试状态和产生难以归因的 diff。更安全的策略是 copy-on-write workspace 或独立 worktree：

```
base snapshot
├─ workspace A → patch A + evidence A
├─ workspace B → patch B + evidence B
└─ workspace C → patch C + evidence C
    ↓
  merge workspace
    ↓
full-system verification
```

合并不是文本拼接。Harness 要检测文件级和语义冲突，决定顺序，并在组合状态上重新运行测试。每个分支单独通过不代表组合后仍通过。

对于数据库、工单和消息系统，使用 resource version、compare-and-swap、事务、幂等键和 effect ledger。读任务可以并发，写任务按资源声明锁或序列化。锁不能由模型用自然语言约定，而应由 runtime 强制执行。

8. 错误如何在团队中传播

多 Agent 的典型错误不是单个 worker 答错，而是错误被社会化：早期错误成为共享前提，后续角色围绕它产生一致但错误的文档。MetaGPT 所说的级联幻觉正是朴素链式协作的问题之一。[MetaGPT](#)

2025 年一项多 Agent 失败研究将问题归为规范与系统设计、Agent 间错位、任务验证与终止三大类，并指出流行 benchmark 上相对单 Agent 的收益可能有限。[Why Do Multi-Agent LLM Systems Fail?](#) 这类结果不应被解读为“多 Agent 无用”，而应提醒我们：新增节点也新增接口，接口比角色名称更决定可靠性。

常见传播机制包括：

- manager 拆错任务，所有 worker 高质量完成了错误子目标；
- worker 把推断标成事实，manager 按多数票强化错误；
- 多个同模型 Agent 共享相同盲点，表面共识并非独立证据；
- 子 Agent 隐去失败，只上报漂亮摘要；
- 循环 handoff 导致责任漂移和预算耗尽；
- verifier 只检查局部结果，没有检查组合不变量。

应对方法包括独立问题分解审查、来源级 provenance、盲化并行、异质模型、反方角色、冲突保留和最终系统级验证。多数票只有在错误近似独立时才有价值；复制同一上下文和模型通常不满足这一假设。

9. Debate、Critique 与 Ensemble

“让多个 Agent 辩论”常被当作通用推理增强，但需要区分三种机制。Ensemble 独立生成候选，再由规则或 judge 选择；critique 让一个 Agent 针对候选找错；debate 允许多轮相互影响。三者成本和风险不同。

受控逻辑推理研究发现，团队内在推理能力与多样性是辩论成功的重要驱动，而顺序、信心可见性等结构参数收益有限；多数压力还可能压制独立纠错。[Can LLM Agents Really Debate?](#) 因此生产系统优先采用“先独立、后比较”：先防止锚定，再暴露候选进行针对性反驳。讨论轮数应由信息增益或分歧收敛决定，不能无限聊到形式共识。

可验证任务通常更适合候选并行 + 外部 verifier，而不是语言辩论。只有当标准含有语义判断、价值冲突或证据解释时，debate 才可能提供额外信息；最终裁决仍应根据完成契约，而不是看哪位 Agent 更善于说服。

10. 调度、背压与预算

多 Agent runtime 本质上是一个带概率 worker 的分布式任务系统。必须处理：队列、公平性、并发上限、速率限制、优先级、超时、取消、心跳、租约、重试、死信和背压。

```
global_budget
├─ orchestration_reserve
├─ worker_pools_by_risk_model_tool
├─ verification_reserve
└─ emergency_reconciliation_reserve
```

不要把全部预算分给探索 worker，最后却没有 token 和时间验证。父 Agent 应在 dispatch 前预留综合与验证预算。动态调度可根据子任务价值、剩余不确定性和边际收益扩缩容：worker 返回重复信息时停止扩展；关键分歧未解决时增加独立路径。

取消必须向整棵委派树传播，但已发生的副作用不能简单“取消”。runtime 需要等待或回读在途 action，执行补偿或标记人工处理。父 run 完成后仍在后台运行的 orphan worker 是成本和安全漏洞。

11. 权限、身份与责任链

每个 Agent 应有独立 execution identity，即使底层由同一模型服务实现。审计记录至少包含 `principal_agent`、`delegated_by`、`capability`、资源范围、策略版本和 `action`。不得用共享管理员 token 让所有 worker 看起来像同一主体。

Manager 对委派行为负责，但不应盲目信任 worker。子结果进入父上下文时仍是不可信输入，尤其当 worker 浏览了网页、issue 或第三方 MCP。合并和提交需要重新经过父级 policy 与 verifier。Handoff 若转移用户交互权，也不能自动转移超出接收者职责的资源权限。

多 Agent 安全还有组合风险：两个单独允许的动作合在一起可能泄露信息或越权。一个 worker 读取私有数据，另一个 worker 向公共系统写入，两者通过共享黑板连接后形成跨域泄漏。信息流策略必须追踪 provenance，而不是只检查单次工具调用。

12. 可观测性：同时看到树和因果链

单 Agent trace 是序列，多 Agent trace 是部分有序图。系统需要同时表示父子关系、消息关系、artifact 依赖和外部 effect：

```
run
├─ delegation span A
│   └─ model/tool spans
│       └─ artifact A
├─ delegation span B
│   └─ model/tool spans
│       └─ artifact B
└─ merge span
    └─ consumes A,B
        └─ verification + commit
```

所有事件使用稳定 run/subtask ID 和逻辑关联，而不是依赖墙钟顺序。并发环境中，时间戳接近不证明因果。指标除成功率外，还应包含有效并行度、重复工作率、消息与 artifact 比例、合并冲突率、委派深度、orphan 数、每次成功成本和关键路径时延。

评测也必须把团队行为作为对象。MultiAgentBench 尝试用 milestone 指标比较 star、chain、tree 和 graph 等协调协议，说明仅看最终答对与否不足以解释协作质量。MultiAgentBench 企业测试还应注入 worker 超时、错误证据、重复消息、部分网络分区、过期 artifact 和恶意子 Agent，观察系统能否降级而不是整体失控。

13. 三类贯穿案例

13.1 仓库软件工程

不要按虚构公司角色拆分，而按可隔离 artifact 拆分：一个 Agent 定位失败，一个在独立 worktree 生成修复，一个构造反例或补充测试，一个做安全审查。主 Agent 保留集成所有权。若多个修改触及同一核心模块，应退化为串行或由单一 owner 实施，避免语义冲突。

每个 worker 交付 patch hash、修改范围和局部测试；合并后在 clean workspace 运行完整完成契约。代码 reviewer 不共享 implementer 的隐藏草稿，只看任务、diff 和证据，以降低锚定。

13.2 企业数据分析

天然可并行的方向包括不同数据源取数、指标口径复核、异常解释和反事实检验。所有 worker 使用固定数据 snapshot，并将 SQL、行数、口径和来源写入结构化结果。Manager 合成结论时保留冲突，不能把不同时间范围或币种的数字直接平均。

数据权限按域分配：财务 worker 不必读取人事明细，外部研究 worker 不接触客户数据。最终报告中的每个数字都回链到查询和 snapshot。

13.3 自我进化 Agent

候选生成、评测和发布必须由不同责任主体完成。多个 mutation worker 并行提出 prompt、skill 或 policy 变体；可信 evaluator 在 held-out 与安全集上评估；selection service 根据预注册规则选择；release controller 做 canary 和回滚。

如果候选 Agent 可以通过委派影响 evaluator，所谓进化就失去独立门禁。委派图、数据血缘和权限必须证明 candidate family 没有接触 held-out 标签，也没有修改评分逻辑。多 Agent 在这里的最大价值不是“群体智慧”，而是实现实验隔离和制衡。

14. 一个最小参考实现

```
function delegate(parent, spec):
    assert spec.objective is bounded
    assert spec.output_schema exists
    assert spec.acceptance_checks not empty

    capability = attenuate(
        parent.capability,
        resources=spec.resources,
        tools=spec.tools,
        ttl=spec.deadline,
        can_delegate=spec.can_delegate
    )

    lease = scheduler.reserve(
        budget=spec.budget,
        parent_budget=parent.remaining,
        cancellation=parent.cancel_token
    )

    child = runtime.spawn(
        snapshot=build_minimal_context(spec),
        capability=capability,
        workspace=create_isolated_workspace(spec.inputs),
        lease=lease
    )

    result = child.await_or_cancel()
    artifact = validate_schema_and_provenance(result)
    checks = verify_subtask(artifact, spec.acceptance_checks)
    return SubagentResult(artifact, checks, child.usage)

function integrate(parent, results):
    reject_unverified_or_expired(results)
    conflicts = detect_semantic_and_resource_conflicts(results)
    if conflicts:
        return RESOLUTION_REQUIRED(conflicts)
    candidate = merge_in_clean_environment(results)
    return parent_completion_gate(candidate)
```

这段伪代码的重点是：委派前缩权和预留预算，执行时隔离，返回时验证 schema 与 provenance，合并时重新检查系统级不变量。Subagent 的“完成”只是父任务的一个候选输入。

15. 设计原则总结

第一，单 Agent 是默认值，多 Agent 需要证明增量价值。第二，按可验证 artifact 和依赖关系拆任务，不按拟人角色拆任务。第三，区分 subroutine、subagent 与 handoff，并显式记录责任所有权。第四，权限随委派衰减，状态与工作区默认隔离。第五，先独立探索，再比较和合并，避免过早共识。第六，所有子结果都携带 provenance、验证和未决项。第七，局部通过不等于组合通过，合并后必须重验。第八，预算覆盖整棵任务树并为验证预留。第九，用分布式系统方法处理取消、重试、背压和孤儿任务。第十，以同成本单 Agent 和多 trial baseline 证明多 Agent 的真实收益。

多 Agent 的成熟标志不是屏幕上出现更多头像，而是组织能够精确回答：为什么要拆成这些执行者，每个执行者看到了什么、被允许做什么、产出了什么证据，冲突怎样处理，以及若其中一个犯错，系统为何仍可恢复。做到这些之后，“Agent 团队”才不再是 prompt theater，而成为可治理的计算拓扑。

第十二章 可观测性、轨迹与评测运营

本章状态：正文初稿 v0.1。

生产 Agent 不能只记录 prompt 与 final answer。真正决定结果的是一次跨模型、工具、环境、策略和人的分布式执行。可观测性的目标不是保存模型私有思维，而是重建可审计的因果链：系统当时看到了什么、采取了什么动作、依据哪个策略、改变了什么状态、用什么证据判断完成。

1. 轨迹是事件图

最小事件模型包括 model turn、tool request、policy decision、approval、execution、observation、artifact、checkpoint、delegation、verification 和 effect commit。事件使用稳定 ID、父子关系与 artifact 引用连接。长输出进入对象存储，trace 保存摘要、哈希和 locator。

```
TraceEvent {
  run_id, span_id, parent_id, type, timestamp
  actor, model, tool, policy_version
  input_refs[], output_refs[]
  state_before, state_after
  cost, latency, status, error_class
}
```

对多 Agent，trace 是部分有序图；对可恢复任务，checkpoint 与 effect ledger 比聊天文本更重要。日志、审计和模型上下文应分开保存：上下文可压缩，运维日志可采样，安全审计则遵循不可篡改和保留策略。

2. 指标分四层

业务层衡量任务价值、人工节省和错误损失；任务层衡量完成率、部分完成、升级率和稳定性；运行层衡量 tool call、重试、上下文、成本、关键路径时延；安全层衡量越权请求、审批、注入、数据流违规和恢复。

平均成功率不足以运营。应按任务族、风险、模型、Harness 版本、工具、仓库规模和上下文长度切片，并同时观察 `pass@k` 与 `pass^k`。[Anthropic Agent Evals](#) 一次最佳表现适合探索能力，连续可靠性才接近生产体验。

3. 失败分类先于优化

失败至少分为：任务规范、上下文选择、推理计划、工具选择、工具执行、环境、权限、验证器、协调、外部依赖和模型能力。若所有失败都记作 `agent_failed`，团队只能凭直觉改 prompt。

归因应连接“最早可纠正事件”而非最后一个错误。测试失败可能源于错误 patch，也可能源于依赖未安装；过早完成可能源于完成契约缺失，而非模型不认真。允许多标签和置信度，保留人工纠正。

4. Eval 是持续运营系统

评测集由生产事故、人工升级、低置信度轨迹、能力边界和安全红队持续补充。Capability eval 探索不会做的任务，regression eval 保护已经会做的任务；高通过率能力题应转为回归题。[Anthropic Agent Evals](#)

每个 task 保存输入快照、reference solution、grader、环境摘要、可见性和退役原因。每次 Harness 变更都与稳定 baseline 做多 trial 对比，报告置信区间、成本和关键切片，而不是只看总分。

5. 在线监控与离线评测闭环

```
production traces
  → privacy filtering
  → failure clustering
  → curated eval candidates
  → independent labeling
  → regression/capability suites
  → candidate harness evaluation
  → canary → production
```

线上反馈不能直接自动成为 prompt 或 memory，否则攻击内容和偶然偏好会固化。采集、筛选、标注和发布之间需要数据治理。用户满意度也不是唯一 reward：Agent 可能通过迎合、隐藏风险或减少必要确认提高短期评分。

6. Trace replay 的边界

模型调用和外部世界不完全可重放。可靠 replay 应固定输入、模型快照、采样参数、工具版本和环境镜像；对不可重放 API 使用录制响应或模拟器。Replay 用于定位差异，不应伪装成绝对复现。

隐私与安全同样重要。轨迹可能包含源码、PII、token 和模型生成的恶意内容。进入分析平台前执行分类、脱敏、租户隔离和最小保留；研究者访问 held-out 与生产数据要审计。

7. 运营仪表盘

一个有用的仪表盘回答：哪些任务失败最多；失败始于哪个层；哪个版本引入退化；自动完成是否真的减少人工总成本；成本上涨来自模型、上下文还是重试；哪些权限请求最常被拒；哪些 verifier 最不稳定。

最终，可观测性不是为漂亮 trace UI 服务，而是为三个闭环服务：事故恢复、工程归因和受控进化。没有可用轨迹，Harness 只能靠 anecdote 进化；没有独立 eval，轨迹优化又容易变成对历史样本的过拟合。

第三篇 产品：当代主流 Harness 的不同答案

第十三章 Claude Code：薄决策环与厚运行时

产品快照截至 2026-08-22。只陈述公开文档与可验证行为，不推断未公开内部实现。

Claude Code 最值得借鉴的不是某个系统提示，而是职责布局：核心 loop 保持简单，把能力放在上下文发现、工具、权限、hooks、subagents、skills、MCP、沙箱和可恢复会话中。官方对 Agent SDK loop 的描述接近“收集上下文—采取行动—验证结果—重复”。[Claude Agent Loop](#)

1. 环境优先

Claude Code 在项目中搜索文件、读取指令、修改工作树并运行命令。项目记忆与路径规则把组织知识放回仓库，而不是永久塞进全局 prompt。[Claude Memory Context window](#) 文档还说明，压缩后不同类型启动上下文具有不同再注入行为，意味着“记忆”实际由多层生命周期组成。[Claude Context Window](#)

设计启示是把上下文当编译产物：稳定前缀、项目规则、当前工作集、工具结果和压缩摘要分别管理。不能把所有内容都称为 memory。

2. 工具与扩展

Claude Code 通过内置工具、MCP、skills、hooks 和 subagents 扩展。Tool search 可不把全部工具 schema 预先注入上下文，而是在需要时检索相关定义，以额外发现回合换取持续的上下文节省。[Claude Tool Search](#)

Hooks 适合确定性策略与集成，skills 适合按需加载过程知识，subagents 适合上下文隔离和并行。把三者混为 prompt 插件会失去权限与生命周期边界。

3. 权限与沙箱

Claude Code 的安全方向从高频逐命令确认转向安全边界内自治。Anthropic 报告其 sandbox 同时限制文件系统和网络，基于 macOS Seatbelt、Linux bubblewrap 与网络代理，并使内部 permission prompt 减少 84%。

[Claude Code Sandboxing](#)

关键原则是：批准精确 capability，而不是信任抽象 Agent。Prompt 负责解释意图，OS 与代理负责不可绕过的边界。外部 MCP、hook 和 skill 仍是供应链入口，沙箱不替代插件审查。

4. 产品取舍

Claude Code 的优势是终端环境贴近工程师真实工作、工具反馈直接、项目约定可版本化，并通过 Agent SDK 把 loop 能力开放给其他应用。它的风险是高度自治 shell 带来的权限面、长会话压缩的信息损失、扩展生态的信任传播，以及产品版本快速变化造成的行为漂移。

Pi 等极简 coding agent 提醒我们：很多能力可以留给 shell 和文件，不必全部内置。Claude Code 的教学价值因此不在“功能越多越好”，而在扩展点如何围绕一个相对薄的循环组织。

5. 企业集成方式

企业不应把 Claude Code 的终端 UI 当平台 API。更合理的是将 Agent SDK/受控进程包装成可替换 runtime adapter，由企业控制面提供身份、任务契约、workspace、策略、凭证、trace 和完成门。

```
enterprise control plane
  → runtime adapter
  → Claude agent session
  → sandbox/tool gateway
  → evidence package
```

平台必须保存供应商无关事件，避免未来自研 runtime 时被 Claude 特有消息格式锁定。Claude 的最终文本只是候选结果，企业 verifier 和 commit authority 仍在外部。

第十四章 OpenAI Codex：协议化 Agent 核心与工程控制面

产品快照截至 2026-08-22。

Codex 展示的是“同一 Harness 核心，多种客户端与执行形态”。公开材料把 agent loop 描述为模型、工具和用户之间的控制器，并解释其如何处理流式事件、工具调用、上下文和循环终止。[Unrolling the Codex Agent Loop](#)

1. App Server 是关键边界

Codex App Server 通过双向 JSON-RPC 把核心能力暴露给 CLI、IDE、桌面和其他客户端，使 UI 不必重新实现 agent loop。[Codex App Server](#) 这是一种重要的平台化：会话、审批、工具事件和状态成为协议对象，而不是终端输出解析。

企业自研 Harness 应借鉴“核心只实现一次，客户端通过版本化协议接入”，同时避免把内部模型 provider 细节泄漏到协议。事件要可扩展，未知事件可向前兼容，命令必须有幂等和恢复语义。

2. 工作树与并行

Codex 的桌面与云形态强调隔离任务、worktree 和并行 Agent。[Introducing the Codex App](#) 代码并行的核心不是多开聊天，而是为每个执行者提供独立工作区，再在 Git 边界合并和验证。

Worktree 解决文件覆盖，不解决语义冲突。多个 patch 合并后仍须在干净环境执行系统级测试，并由单一 owner 决定提交。

3. 策略层

开源 Codex 包含 sandbox、approval 与 exec policy。ExecPolicy 使用 allow、prompt、forbidden 的命令前缀规则，并允许规则附测试样例。[Codex ExecPolicy](#) 这说明持久批准应成为可测试的策略代码，而不是模糊的“始终允许”。

企业扩展还需加入身份、资源、数据分类和跨域流动；命令字符串规则只是其中一层。

4. Harness Engineering

OpenAI 将自身实践概括为 harness engineering：让仓库结构、测试、文档、日志和工具对 Agent 可读、可操作、可验证。[Harness Engineering](#) 这改变了平台投资方向。提升 Agent 不只是在 prompt 上打补丁，也包括缩短环境反馈回路、提高错误可诊断性、把隐性规范变成可执行检查。

5. 取舍与借鉴

Codex 的优势是开源核心、协议化 App Server、工作区隔离和系统策略。风险包括客户端/服务器协议演化、云与本地能力差异、并行任务的成本与合并复杂度，以及模型与 Harness 同厂优化造成的可移植性幻觉。

企业最应借鉴的是控制面与执行面的协议化，而不是照搬工具名称。将 Codex 作为 runtime 时，外部平台继续拥有任务合同、租户身份、数据策略和最终证据；将来自 Codex 的事件映射为 canonical trace，未来即可替换为其他 runtime。

第十五章 Cursor：IDE 原生上下文与云 Agent

产品快照截至 2026-08-22；厂商使用数据只作为其用户群观察，不外推全行业。

Cursor 的代表性在于把 Harness 嵌入开发者交互面：编辑器状态、选择区、诊断、终端和代码索引成为上下文来源；同一产品又向云端长任务和隔离 VM 延伸。它展示了 Harness 不只是后台 runtime，也是人与 Agent 共享注意力和控制权的界面。

1. 动态上下文发现

Cursor 公开描述其从大量静态上下文与强约束，转向由模型主动发现所需信息。长工具结果可以写入文件，模型按需读取；工具 schema 可以动态发现；终端状态也可通过统一文件式接口访问。[Dynamic Context Discovery](#)

这与“最小充分工作集”原则一致：不要预判并注入全部内容，而要提供廉价导航、搜索与回读。但动态发现增加工具轮次，若索引、文件命名或错误反馈差，模型会在探索中浪费预算。

2. Harness 与模型共同适配

Cursor 明确讨论了不同模型需要不同工具描述、约束和交互设计，并通过线上与离线评测持续改进 Harness。[Continually Improving Our Agent Harness](#) 这反驳“一套系统提示适配所有模型”的设想。

企业 provider abstraction 因而不应只统一 API。Canonical action contract 可以稳定，但 model-facing tool view、上下文布局和错误呈现需要按模型 profile 编译。可移植性发生在控制面，性能优化发生在适配层。

3. 云 Agent 的环境工程

Cursor 对 cloud agents 的总结强调预构建环境、VM checkpoint/restore/fork 和专用 computer-use 能力。[Cloud Agent Lessons](#) 长任务的速度很大程度取决于环境启动、依赖缓存和可恢复性，而不只是生成速度。

Checkpoint 还支持从同一状态派生多个候选，但必须区分环境快照与任务真相：外部服务仍会变化，凭证与租约可能过期，恢复后需重新验证策略和 freshness。

4. IDE 人机协作的优势与风险

IDE 内 Agent 能显示 diff、引用诊断并让开发者随时接管，适合高频、局部和互动式任务。风险是隐式上下文过多：打开文件、剪贴板、终端和索引可能包含敏感数据；频繁自动接纳可能降低审查质量；本地与云端的权限边界也容易被界面统一感掩盖。

产品指标如自动接纳率、工具调用深度和上下文规模能展示趋势，但不能直接代表正确率。接受可能来自信任、疲劳或低风险任务。企业评估仍需以合并后缺陷、返工和交付周期为准。

5. 架构启示

Cursor 提供三条可迁移经验：交互面本身是 Harness 的一部分；模型 profile 应控制上下文和工具适配；云 Agent 的竞争力来自环境基础设施。企业平台若只提供一个聊天框和通用 API，即使模型相同，也难复制 IDE 原生 Agent 的能力。

第十六章 DeepSeek Harness：可组合运行时与进化载体

产品快照截至 2026-08-22。DSH 官方仓库仍应按 developer preview 看待。

DeepSeek Harness (DSH) 最重要的贡献，不是已经实现了一个可信的自主进化 Agent，而是把 Harness 本身设计成可组合、可替换、可卸载的运行时。它让“运行时结构可以变化”成为一等能力，也因此把自我进化的安全与验证问题推到台前。

1. Cordis 插件树

DSH 运行实例建立在 Cordis 插件树上。模型适配、工具、持久化、默认 loop 等都可作为插件装配。[DSH Architecture](#) Cordis 用 context、service dependency、typed event/waterfall 与 effect ownership 管理组件生命周期。[Cordis Primer](#)

所谓时间可组合性，是组件卸载时能撤销由其注册的 effect；空间可组合性，是组件按依赖和所在 context 激活。相比“启动时注册一堆全局回调”，这种设计更适合长生命周期、热装配和实验变体。

2. 分层配置与 scope

DSH 通过 profile、bundle、用户 patch、home patch 和临时 overlay 组合配置，并区分 host scope 与 agent scope。不同会话可拥有不同模型、工具、persona、压缩策略和扩展，同时共享宿主服务。

这给企业平台一个有价值的方向：配置不是一个巨大 JSON，而是带来源、优先级、生命周期和撤销语义的 patch。每个实验变体可以绑定 scope，避免修改污染所有租户。

3. Tool Runtime 与 Code Mode

DSH 的工具运行时支持将工具生成 SDK 视图，通过 `run_code` 让模型组合调用；嵌套调用仍回到受控工具管线，只有显式打印或返回的数据进入外层上下文。[DSH Code Mode](#)

这兼具 CodeAct 的表达力与集中 policy enforcement。风险在于生成 SDK、sandbox 和嵌套调用的语义复杂度，必须防止通过解释器、网络或输出通道绕过工具策略。

4. 可变不等于会进化

一个可热替换插件的系统只是 evolution substrate。可信进化还需要轨迹采集、失败归因、候选生成、独立评测、统计门禁、canary、回滚和审计。Agent 不能修改给自己评分与授权的根信任。

近期 Self-Harness、Gated Semantic Quality-Diversity、Living-Harness 和 Hierarchical Self-Improvement 等工作分别探索 Harness 候选生成、确定性门禁、经验状态图和层级改进，但仍是快速发展的研究方向。[Self-Harness](#)

5. 企业取舍

DSH 的优点是微内核式组合、生命周期所有权、scope 与配置 patch，适合做可实验的 Agent runtime。风险是抽象学习成本、插件依赖图、动态装配的可预测性和开发预览阶段的稳定性。

企业可借鉴 Cordis 的 effect ownership 与配置 provenance，而不必立即采用整个实现。若把 DSH 接入平台，应把可演化插件域与不可变控制面隔开：身份、策略、evaluator、审计与发布控制器不能由任务内 Agent 自行替换。

第十七章 OpenHands: Agent 与 Runtime 分离的开放架构

产品快照截至 2026-08-22。

OpenHands 被选为第五主案例，不是因为它一定拥有最大用户规模，而是其开放代码、论文传统和 Agent/Runtime 分离对企业架构最具教学价值。它补足了四个商业产品公开实现透明度不足的问题。

1. Action—Observation—Event

OpenHands 用 action 表示 Agent 意图，用 observation 表示环境反馈，并通过事件流连接会话。[OpenHands Paper](#) 这个协议边界允许替换 Agent 策略、模型和 Runtime，也允许记录、重放与插入策略。

关键不是类名，而是模型不直接操作宿主。Runtime 接收规范化动作，在受控环境执行并返回结构化观察。终端输出、文件变化、浏览器状态和错误都成为事件。

2. Runtime 生命周期

OpenHands Runtime 可以运行在 Docker 或远程环境中，负责初始化、执行、文件传输和 teardown。

[OpenHands Runtime Architecture](#) 这把高风险计算面从 Agent server 分离，也为企业替换 Kubernetes、VM 或专用沙箱提供接口。

远程 Runtime 并不自动安全。镜像供应链、网络、凭证、租户隔离和 artifact 导出仍需控制面治理。协议只提供插入控制的机会。

3. 开放平台的价值

开放实现允许研究者比较不同 Agent、模型与工具，并在 SWE-bench 等环境中复现。它也暴露生产化成本：事件 schema 演化、Runtime 兼容、部署复杂度、持久化与 UI 都需要持续工程。

OpenHands 的设计比“一个 Python while loop + shell”更适合作为企业参考，是因为它天然支持执行面的独立扩缩、故障隔离和审计。但企业仍需要补充统一身份、策略即代码、证据包和供应商 runtime adapter。

4. 与其他案例的互补

Claude Code 展示终端产品与扩展生态，Codex 展示协议化核心与多客户端，Cursor 展示 IDE/云环境，DSH 展示可组合插件树；OpenHands 则把 Agent 与计算 Runtime 的边界公开化。五者共同说明，Harness 不是单一框架，而是一组控制面和数据面职责。

5. 企业采用方式

最稳妥的采用不是 fork 全部代码并深度改造，而是把 Runtime protocol、event model 和 workspace lifecycle 作为可替换组件接入。上层平台生成 canonical task，映射为 OpenHands session；下层接收 evidence package，再由企业完成门决定提交。

当未来自研 Agent loop 时，可以保留 Runtime 与控制面，只替换决策策略。这正是开放架构的长期价值。

第十八章 横向比较：不同 Harness 的答案

比较快照截至 2026-08-22。矩阵描述公开能力与设计重心，不等同于质量排名。

五个主案例并非五套互斥架构，而是对相同责任作出不同取舍。

维度	Claude Code	Codex	Cursor	DSH	OpenHands
主要交互面	终端/SDK	CLI、IDE、App、云	IDE、云	Runtime/CLI	Web/SDK/研究平台
核心重心	薄 loop、扩展与安全	协议化核心、工作树、策略	动态上下文、IDE 与 VM	插件树与生命周期	Agent/Runtime 分离
上下文	项目规则、压缩、tool search	会话与 compaction	动态发现、索引、文件化结果	可替换策略	事件与观察
执行隔离	OS sandbox	sandbox/worktree/cloud	本地与 cloud VM	sandbox service	Docker/remote Runtime
扩展	hooks、skills、MCP、subagent	tools、MCP、App Server	MCP、IDE/云能力	Cordis plugins	Agent/Runtime/tool 扩展
独特价值	开发者终端闭环	多客户端控制面	交互原生性	可变运行时载体	开放协议边界

1. 薄与厚不是优劣

Pi 代表极薄 Harness：少量工具、依赖 shell 和文件、刻意不内置复杂 plan、permission UI 或多 Agent。这提醒我们，功能越多不必然越可靠。相反，企业场景要求身份、审计、策略和恢复，厚控制面又不可避免。

合理分层是：模型面对的动作面保持小而清晰，运行时内部可以很厚。复杂度应服务于确定性边界，而不是把更多抽象暴露给模型。

2. 本地与云

本地 Agent 接近开发者环境、启动快、交互自然，但宿主秘密和环境漂移风险高。云 Agent 易隔离、并行和恢复，却有环境准备、数据上传、凭证代理与成本问题。现代产品通常走向混合：控制面统一，本地与云作为不同 execution profile。

3. 开放与闭源

开源可验证协议、策略和沙箱实现，闭源产品可能拥有更成熟模型适配和运维数据。企业选择不应只看许可证，而要看可导出的 trace、artifact、策略控制、数据边界、版本可固定性和退出路径。

4. 共同收敛

五者正在共同收敛到：持久会话、按需上下文、结构化工具、隔离执行、审批策略、MCP/扩展、多 Agent、可观测和验证。差异逐渐从“有没有工具调用”转向每个层的质量与组合方式。

5. 选择原则

交互式个人 coding 优先考虑 IDE/终端体验；后台并行任务重视云工作区和协议；强定制企业平台重视开放 Runtime、策略与事件；进化研究重视可组合配置和评测接口。没有一个产品应同时作为组织的身份源、策略根、证据库和唯一执行 runtime。

因此企业架构的目标不是选出永久赢家，而是定义稳定的 canonical contracts，让五类 runtime 都能被接入、比较和替换。

第四篇 进化：Agent 如何从轨迹中变得更好

第十九章 进化不是自我修改：目标函数、数据与治理

Agent 进化常被描述成“它会修改自己”。这种说法隐藏了最重要的问题：谁定义更好，谁提供数据，谁评价候选，谁有权发布，失败如何回滚。工程上，进化是一个受控优化系统，而不是自治主体获得无限写权限。

1. 四个层次

本书把进化分为：任务内策略适应；跨任务记忆与 skill；Harness 的 prompt、工具、路由和工作流变化；模型参数变化。越往后影响面越大、反馈越慢、治理成本越高。

task-time repair	→	memory/skill	→	harness release	→	model training
minutes		days		weeks		weeks/months

不要用模型训练解决本可由工具 schema 修复的问题，也不要吧短期上下文摘要冒充长期学习。

2. 优化对象与根信任

系统先声明 mutable surface：哪些 prompt、retriever、tool view、policy 参数、workflow 或 memory 可产生候选。Evaluator、held-out 数据、权限根、审计和 release controller 默认不可由候选修改。

如果 Agent 同时修改实现和评分器，分数上升没有意义。自我进化的第一原则是评价独立性。

3. 数据不是天然经验

生产轨迹包含成功、偶然成功、失败、攻击、用户妥协和环境噪声。进入经验库前要脱敏、归因、去重、标注任务分布和结果证据。只学习被用户接受的答案会产生选择偏差；用户可能没有检查。

4. 多目标而非单分数

目标至少包含正确性、稳定性、安全、成本、时延、人工负担与可解释性。优化单一通过率容易导致更长轨迹、更多权限或测试投机。采用 Pareto frontier 与硬约束：安全回归不允许被平均收益抵消。

5. 最小可信闭环

observe	→	attribute	→	propose minimal change		
	→	isolated multi-trial eval	→	statistical gate		
	→	canary	→	monitor	→	promote/rollback

Self-Harness 等近期工作说明冻结模型时，Harness 改进也可能产生显著收益；但论文结果不等于生产自治许可。Self-Harness 组织需要可重放证据和发布治理。

6. 能力边界

Harness 可以减少接口摩擦、提供搜索与验证、扩大推理预算，却不能无限补偿模型缺少的知识和推理能力。Hierarchical Self-Improvement 报告的边界性结果提醒：在超出基础能力的任务上，结构优化可能没有提升。
HSI

进化项目应建立对照：模型升级、Harness 变化、环境变化和数据污染分别测量。否则组织会把供应商模型进步误认为自研 Harness 学会了进化。

第二十章 任务内进化：搜索、反思与验证—修复

任务内进化不改变长期系统版本，而是在一次 run 中根据反馈调整计划、候选和资源。它是最安全、反馈最快的一层，也最容易被误称为“自我学习”。

1. Reflexion 的贡献与边界

Reflexion 将失败反馈转成语言记忆，在后续 trial 中影响行为，不更新模型权重。Reflexion 它证明文本反馈可形成短期策略改进，但反思是否正确仍依赖 evaluator。让同一模型自由写“教训”可能固化错误归因。

2. 搜索不是无限重试

候选搜索可以采用 best-of-N、树搜索、分支工作区或多 Agent 并行。每个分支必须有不同假设、预算和停止条件；重复相同 prompt 只是在采样。选择由外部 verifier 进行，不能按语言自信度。

3. 验证—修复循环

```
candidate → deterministic checks
├─ pass → completion gate
├─ diagnostic failure → minimal repair context
├─ flaky/ambiguous → independent review
└─ no progress/budget → escalate
```

Harness 只回传修复所需诊断，避免逐轮泄漏 held-out。使用错误签名检测循环；连续两次没有减少失败集合时换假设，而不是继续局部补丁。

4. 动态 workflow

任务内可以根据风险与不确定性增加搜索、reviewer 或测试，但 runtime 仍执行预算与权限上限。模型可以提议新步骤，不能自行取消强制检查。

5. 三类案例

代码 Agent 在独立 worktree 生成多个 patch，由测试和静态分析选；数据 Agent 对异常结论生成替代查询并对账；自进化实验中的候选生成器根据失败簇提出最小 mutation。共同点是变化留在 run scope，结束后不自动污染全局系统。

任务内进化的成熟指标不是“思考轮数”，而是单位成本下错误集合是否收敛、是否避免重复副作用、是否保留可解释的候选淘汰记录。

第二十一章 跨任务经验化：Memory、Skill 与策略库

跨任务进化把一次 run 的信息带到未来。它比任务内修复更有杠杆，也更容易形成持久污染。核心问题不是“记住更多”，而是哪些经验值得固化、在什么条件下检索、何时过期、谁能撤销。

1. 四类持久对象

事实记忆保存相对稳定的领域信息；情景记忆保存任务、动作与结果；程序记忆以 skill、脚本或 SOP 表达做法；策略统计保存某类选择的效果。四者拥有不同验证和 TTL，不能都塞进向量库。

Voyager 的 skill library 展示了把验证过的可执行技能积累并在未来复用的路线。Voyager Claude 的项目规则则展示由人维护、随仓库版本化的程序知识。前者偏自动发现，后者偏组织治理。

2. 写入门比检索更重要

自动记忆必须满足来源可信、结果已验证、可泛化、无秘密、与已有项不冲突。一次成功不能证明因果。候选 memory 先进入隔离区，经多任务验证和人工/策略批准后晋级。

```
trace → candidate lesson → evidence linkage  
→ dedupe/conflict → held-out reuse eval → publish with TTL
```

3. 检索与适用条件

每条经验带适用范围、前置条件、反例、版本和置信度。检索器不仅按语义相似，还按环境、工具版本、租户和 freshness 过滤。过期 skill 应失败关闭，而不是悄悄运行旧命令。

4. Skill 供应链

Skill 可能包含指令、脚本和资源，等同于可执行依赖。需要 owner、版本、签名、权限 manifest、测试、变更评审和撤销。Agent 自动生成 skill 只能进入候选 registry，不直接成为全局能力。

5. 遗忘与纠错

记忆系统必须支持 provenance 查询、降权、失效和删除。用户纠正不是简单追加相反文本，而要定位受影响的记忆和派生 artifact。组织还需满足数据删除与租户隔离。

跨任务进化成功的标准，是未来任务在稳定成本和安全约束下改善，并能证明改善来自哪条经验；不是 memory 条目持续增长。

第二十二章 Harness 进化：Prompt、工具、上下文与 workflows

Harness 进化直接修改模型所处的决策环境，通常比训练模型便宜、上线快。可变对象包括 system instruction、tool schema、上下文选择器、压缩器、router、retry、workflow、sandbox profile 与模型 profile。

1. 先归因再变异

工具选择失败可能来自描述、参数、返回噪声或模型能力。盲目追加 prompt 会形成不可维护的规则堆。每个 mutation 应对应失败簇和因果假设，例如“工具目录过大导致选择错误”，候选则是动态 tool discovery，而非泛化提醒。

2. 最小变更原则

候选表示为版本化 patch：

```
HarnessMutation {
  base_version
  target_component
  hypothesis
  patch
  expected_gain
  risk_surface
  eval_plan
}
```

一次只改变尽量少的因素，便于归因和回滚。涉及多个组件的组合优化可在单变量证据后进行。

3. 质量多样性

只保留最高平均分候选会收敛到单一策略，并可能牺牲某些任务族。Gated Semantic Quality-Diversity 将候选多样性与确定性门禁结合，强调把生成交给模型、计量和显著性检验交给代码。[Gated Semantic QD](#)

企业可以维护按任务域、风险和模型区分的多个 Harness profile，而非追求一个全局最优 prompt。

4. 四组门禁

正确性门检查能力与回归；安全门检查权限、注入和信息流；运营门检查成本、时延和稳定性；治理门检查可解释性、所有者与回滚。任何硬门失败都不能被平均收益抵消。

5. 发布

候选先 shadow，再小流量 canary，随后按切片晋级。运行时记录完整版本组合：模型、prompt、tools、retriever、policy、sandbox 与 evaluator。回滚必须能恢复组合，而不仅是 prompt 文本。

DSH/Cordis 为动态装配提供了优雅载体，但 evolution controller 应在插件树外部。可修改性与评价权分离，是 Harness 进化从 demo 走向生产的分界线。

第二十三章 模型进化：轨迹蒸馏、偏好与强化学习

当问题跨任务重复出现、无法仅靠接口和上下文修复，且有足够高质量数据时，才考虑模型参数进化。Harness 在这里既是数据生成器，也是评测与部署容器。

1. 轨迹不等于训练样本

生产轨迹包含冗余探索、工具错误、秘密、偶然成功和环境依赖。训练前需要结果验证、步骤归因、脱敏、去重、难度与任务分布标注。只蒸馏成功轨迹可能教会模型隐藏失败；还需保留纠错和负例。

2. SFT 与蒸馏

SFT 适合稳定格式、工具协议和高质量行为模式。强模型或昂贵 Harness 可产生候选轨迹，由 verifier 过滤后训练更小模型。但学生模型可能模仿文本表面而未获得环境适应能力，因此必须在真实 Harness 中评测。

3. 偏好优化

成对比较可训练模型偏好更安全、简洁或可验证的轨迹。Preference 数据应基于结果与 rubric，而非只由同族模型 judge；否则把 judge 偏差蒸馏进模型。

4. RL 与可验证奖励

可执行任务提供测试、约束和环境结果作为奖励，适合 RLVR。风险是 reward hacking：修改 evaluator、硬编码 visible test、泄漏 held-out 或通过更危险权限取巧。Reward 必须由隔离控制面重算，并把安全、成本纳入约束。

5. 模型—Harness 2×2 归因

比较旧模型/新模型与旧 Harness/新 Harness 四个组合，才能判断收益来自哪里：

	旧 Harness	新 Harness
旧模型	baseline	Harness gain
新模型	model gain	combined

多 trial 和任务切片还能发现交互效应：新 Harness 可能只适配某模型。模型发布后继续保留旧版本回归，避免把基础能力变化误判为环境问题。

模型进化的门槛高于 Harness 进化：数据、训练、模型安全和部署都需要独立治理。它不是每个企业平台的必建能力；很多组织更适合先建立高质量轨迹与 eval，再与模型提供方或专门训练平台合作。

第二十四章 受控进化闭环：门禁、灰度、回滚与反投机

前三章分别讨论可变对象，本章把它们组成生产闭环。可信进化不是 Agent 在运行时修改自己，而是候选系统在不可变治理框架下接受实验。

1. 双平面架构

```
immutable governance plane
  identity / policy root / eval registry / held-out vault
  release controller / audit / rollback

evolvable plane
  prompts / skills / memory / tool views / workflows
  model profiles / candidate plugins
```

候选平面只能提交 proposal，没有自行晋级权限。治理平面也不接收候选生成的自报分数，而在隔离环境重算。

2. 实验协议

每次实验预注册目标、主要指标、硬约束、任务集、trial 数、停止规则和允许风险。基线与候选随机交错运行，减少时间和环境漂移。报告总体与关键切片、置信区间、成本、失败簇和完整性告警。

3. 防 Reward Hacking

Evaluator 只读隔离，held-out 不进候选上下文；记录文件与网络访问；Agent 报告分数与可信重算对账；测试文件、metric 代码和数据 hash 纳入 evidence package。EvilGenie 与 SpecBench 说明 visible test 通过并不足以证明真实目标。[EvilGenie](#)

4. Canary 与回滚

发布从 shadow、内部、低风险租户到广泛流量。Canary 采用版本粘性，避免同一任务中途切换。异常触发自动停止新任务，进行中任务按风险完成或暂停。回滚同时恢复 Harness bundle、模型 profile、memory snapshot 和 policy compatibility。

5. 组织责任

产品 owner 定义价值，领域专家维护任务，安全团队定义硬门，平台团队维护 runtime，独立评测方管理 held-out，发布责任人批准晋级。小组织可一人多角，但系统权限仍分离。

6. 进化账本

保存 lineage：父版本、mutation、数据、评测、选择原因、canary、事故和退役。这样可回答“为何变好”“谁批准”“哪些任务退化”“如何回去”。没有 lineage 的自动优化只是不可审计配置漂移。

成熟系统的目标不是最大更新频率，而是最大可信学习率：每次变化都能从证据中学习，同时把错误候选限制在可恢复的爆炸半径内。

第五篇 实践：下一代企业 Harness

第二十五章 三个贯穿案例的端到端设计

1. 仓库级软件工程

入口把 issue 编译为 completion contract：目标、允许目录、兼容不变量、测试和 PR 证据。控制面创建固定 commit 的 worktree，runtime adapter 启动 Claude Code、Codex 或自研 Agent。Agent 搜索、修改和测试；高风险依赖安装或网络访问经策略门。

候选 patch 被 seal，在 clean workspace 执行 fail-to-pass、pass-to-pass、lint、安全和变更范围检查。独立 reviewer 只看任务、diff 和证据。通过后生成 PR；merge 仍由人或发布策略批准。失败轨迹按上下文、工具、代码、环境或 verifier 归因，进入 eval 候选池。

2. 企业数据分析

任务合同固定指标口径、数据快照、时间、币种、允许来源和交付格式。Planner 将取数、对账、解释和反证分解；每个 worker 使用最小权限短期凭证，敏感数据不进入外部模型上下文。

SQL、参数、行数、数据 hash 和图表源成为 artifact。数字由独立查询与总额对账验证，文字结论由 rubric/model/分析师检查。最终 evidence package 能让另一位分析师重建报告。任何 freshness 变化都标记，不静默混合快照。

3. 自我进化 Agent

Observability 聚类一段时间的失败，提出“工具目录过大造成选择错误”的归因。Mutation workers 生成动态发现、描述改写和模型 profile 三类候选。Evaluator 在 held-out、回归、安全与成本集上多 trial 运行；候选无权读取标签或修改 evaluator。

统计门选择非劣且显著改善的 profile，先 shadow 后 canary。监控选择错误、任务成功、token 和权限请求。若关键切片退化，release controller 回滚 bundle 并记录 lineage。整个闭环没有让生产 Agent 直接改写自身。

4. 共用骨架

```
intent → contract → identity/workspace → runtime  
→ actions/effects → candidate → independent verification  
→ approval/commit → evidence → telemetry → eval/evolution
```

三例的差异在工具、数据和风险，骨架相同。平台化价值来自复用任务、身份、策略、证据、trace 和发布，而不是强迫所有 Agent 共享一种内部思考方式。

第二十六章 下一代企业 Harness 参考架构

参考架构的目标不是重新实现每个 coding agent，而是在 Claude Code、Codex、OpenHands、DSH 与未来自研 runtime 之上建立稳定控制面。

1. 六层结构

Experience	IDE / Web / CLI / API / workflow
Control Plane	task contract / scheduler / identity / policy / approval
Agent Runtime	vendor adapter / loop / context / delegation
Execution Plane	workspace / sandbox / tool gateway / credential broker
Evidence Plane	artifacts / trace / verifier / effect ledger
Evolution Plane	eval registry / mutation / experiment / release / rollback

体验层不拥有任务真相；控制面生成 canonical task。Agent runtime 可替换。执行面负责真实副作用。证据面独立于聊天历史。进化面只能发布经门禁的版本。

2. Canonical contracts

平台至少定义 Task、Action、Observation、Artifact、PolicyDecision、Checkpoint、Delegation、VerificationResult 和 EvidencePackage。Vendor adapter 在 canonical event 与产品协议之间映射，保留原始 payload 引用以便诊断。

```
RuntimeAdapter {
  capabilities()
  start(task, workspace, policy_profile)
  stream_events(run_id)
  approve_or_deny(request)
  checkpoint(run_id)
  cancel(run_id)
  collect_artifacts(run_id)
}
```

不要强求所有 runtime 暴露同样的内部 reasoning。统一可观察动作与结果即可。

3. 身份与租户

User、platform、runtime、subagent、tool 和 external service 都有独立身份。授权以 capability lease 表达，绑定租户、资源、动作和 TTL。Credential broker 在执行时注入短期凭证，模型不看到 secret。

4. Durable execution

Run state 持久化，工具副作用记录幂等键和 outcome。Worker 崩溃后从 checkpoint 恢复；对不确定外部结果先 reconcile。Scheduler 管理预算、优先级、并发和取消树。

5. Policy 与 sandbox

策略在 action commit 时评估，输出 allow、deny、approval 或 constrained allow。Sandbox 同时限制文件、网络、进程和资源。Policy profile 与 sandbox image 都版本化并进入证据包。

6. Evidence-first completion

Runtime 只能提交候选。Verifier service 在隔离环境运行完成契约，生成签名结果；commit controller 再执行外部提交。此边界使不同 runtime 可公平比较，也阻止供应商 Agent 自报完成。

7. 演进边界

Prompt、tool view、retriever、skill 和 workflow 可进入候选；identity root、policy root、held-out、审计与 release controller 不可由候选修改。所有版本组合进入 lineage registry。

8. 非功能要求

以租户隔离、可用性、恢复时间、trace 完整性、最大副作用、成本预算和数据驻留定义 SLO。Agent 成功率不是唯一 SLO；错误完成的代价通常远高于明确失败。

这套架构允许企业先集成现有 runtime，再逐步替换上下文、loop 或工具层，而无需重建治理与证据系统。

第二十七章 Agent SDD：规范驱动的任务与发布

Agent SDD (Specification-Driven Delivery) 不是要求所有请求先写长文档，而是把关键意图转换成可执行、可版本化的契约，使自治执行有明确边界。

1. 规范层级

业务规范描述价值；任务规范定义交付物与不变量；工具契约定义动作；策略规范定义权限；验证规范定义证据；发布规范定义谁能让效果生效。自然语言可作为入口，但最终关键字段应结构化。

2. 从意图到合同

Agent 可协助澄清和生成 contract draft，用户只确认高材料性歧义。低风险探索允许渐进完善，高风险任务必须在执行前冻结关键不变量。

```
intent → ambiguity detection → contract draft  
→ authority confirmation → executable checks → run
```

运行中发现新事实可以提出 contract amendment，但不能由执行 Agent 单方面降低验收标准。每次修改保留 diff 和批准者。

3. Specification as environment

规范应贴近权威状态：代码规则进入仓库，数据口径进入 semantic layer，API 约束进入 schema，安全要求进入 policy-as-code。只写在 prompt 的规范难以测试和复用。

4. 发布门

候选 artifact 与 contract version 绑定；verifier 证明检查结果；waiver 明确风险和到期；commit controller 负责合并、发送或部署。模型停止与发布完全解耦。

5. 规范债务

过细规范会把 Agent 退化成昂贵工作流，过粗规范会产生假完成。通过生产失败持续调整边界：把重复、可确定的隐性要求转为 schema、测试或策略，把真正需要判断的部分保留给模型和人。

Agent SDD 的价值是让平台可比较不同模型与 Harness：相同合同、相同环境、相同完成门，差异才可归因。

第二十八章 成熟度模型与 Build-vs-Buy

1. 五级成熟度

等级	特征	主要风险
L0 对话增强	单轮/简单工具	无完成证据
L1 受控执行	workspace、基础权限、日志	恢复与策略薄弱
L2 可验证任务	contract、verifier、artifact	评测覆盖不足
L3 平台化运行时	多 runtime、durable、租户策略	复杂运营
L4 受控进化	轨迹、实验、canary、rollback	优化投机与治理

成熟度按最弱关键层判断，不能因 UI 漂亮或模型强而跳级。没有独立完成门的多 Agent 平台仍可能停在 L1。

2. 买什么

优先购买快速变化且有规模效应的能力：前沿模型、成熟 coding runtime、浏览器/计算沙箱、通用连接器。评估数据边界、可固定版本、trace 导出、权限控制、SLA 和退出成本。

3. 自建什么

企业差异化和责任不可外包的部分应自建或牢牢控制：任务合同、身份映射、业务策略、凭证代理、领域 verifier、证据与审计、eval 数据、发布门和 runtime abstraction。

4. 何时自研 runtime

只有当任务规模足够、现有产品在关键接口受限、定制收益可量化、团队能承担安全与运维时，才自研 loop/runtime。模型调用和 shell 很容易，durability、兼容、安全、恢复、评测和生态才是长期成本。

5. 决策矩阵

按任务匹配度、控制力、透明度、数据风险、总成本、可替换性和演进能力评分。POC 必须使用真实任务、真实权限与完整失败成本，不用厂商 demo 或单一 benchmark。

推荐战略通常是“买 runtime，建控制面，保留替换权”；随着组织成熟，再选择性内化上下文、工具或 loop。

第二十九章 从接入现有 Agent 到自研运行时

阶段一：封装而非散接

为 Claude Code、Codex 或其他 Agent 建立 adapter，统一 task、event、artifact、approval 和 cancel。所有调用经过平台身份、workspace 与策略，不允许业务团队直接分散保存 token 和脚本。

阶段二：外置完成与证据

先把 verifier、evidence package 和 commit authority 放到 runtime 外。这样即使更换 Agent，业务正确性与审计不随供应商迁移。

阶段三：统一执行面

建立企业 sandbox、tool gateway、credential broker 和 artifact store。供应商 runtime 只决定动作，不直接持有生产凭证。对无法适配的功能保留专用 execution profile。

阶段四：建立评测基线

从真实任务形成 capability、regression 和 safety suites，以相同合同比较不同模型/runtime 的成功、稳定、成本和人工负担。没有基线，自研无法证明价值。

阶段五：逐层替换

先替换最具差异化的 context compiler、tool view 或 workflow，再考虑 loop。每次只替换一层，保留 2×2 对照和快速回滚。不要一次重写 UI、runtime、sandbox 和 eval。

阶段六：引入受控进化

当 trace、归因和 eval 稳定后，才自动生成候选 Harness 变更。发布仍由外部治理平面控制。模型训练是更后的选择。

迁移反模式

直接解析终端彩色输出；把厂商消息结构当领域模型；把供应商“完成”映射为业务成功；共享宿主凭证；无版本本地自动更新；只比较 token 价格；在没有 eval 时宣布自研更强。

最终目标不是完全摆脱供应商，而是让供应商成为可替换能力组件。平台拥有任务定义、权力边界、证据和学习数据，才拥有长期架构主动权。

第三十章 展望：Harness OS、Agent 组织与持续进化

未来 Harness 会更像 Agent 的操作系统：调度概率执行者，管理上下文与能力，隔离计算，记录副作用，验证结果，并在多版本之间安全演进。这个比喻的价值在职责，不在复刻传统 OS API。

1. 模型与 Harness 共同设计

工具使用、上下文读取和压缩将越来越进入模型训练；Harness 又会按模型 profile 动态编译接口。通用 provider API 仍存在，但性能前沿来自协同设计。企业需要稳定 canonical contract 与可变 model-facing view 的双层结构。

2. 从应用到 Agent 组织

Agent 将跨越单个聊天，成为有身份、预算、工作区和持续责任的数字执行者。多个 Agent 组成按任务动态生成的组织，但组织图必须由合同、权限和 artifact 定义，而非角色扮演。

人类工作从逐步操作转向定义目标、处理例外、维护规范和审查证据。若企业流程仍只有口头约定和不可观测系统，Agent 只会放大组织熵。

3. 环境成为主要护城河

模型能力趋同后，差异来自企业是否让数据、工具、日志、测试和审批对 Agent 可用且安全。Harness engineering 本质上也是组织的“可机器操作化”工程。

4. 持续进化的现实形态

短期内可信进化更可能是自动提案、自动评测、人或策略批准，而不是生产 Agent 任意改写自己。随着 evaluator、形式约束和沙箱成熟，可自动晋级的范围逐渐扩大，但根信任仍保持外部。

5. 新风险

长时自治、跨 Agent 信息流、插件供应链、评测污染、经济型 DoS、自动生成并固化错误 skill，以及多个 Agent 合谋或相互强化偏差，会成为主要治理议题。安全评价必须覆盖整个 Harness，不再只测裸模型拒绝率。

6. 最终判断

Agent 的竞争不只是谁拥有最强模型，而是谁能以更少上下文、更小权限、更低协调熵，在真实环境中持续产生可验证结果，并把失败转化为受控改进。Harness 将从脚手架变成企业 AI 劳动力的制度与基础设施。

最值得建设的不是一个“永远正确的自治 Agent”，而是一套知道自己何时不确定、能证明完成、能安全失败、能从证据中改善且始终可被人类治理的系统。

附录

附录 A：语言无关核心接口

```
Task {id, tenant, contract_version, input_refs, risk, budget}
Action {id, actor, type, normalized_args, resource, provenance}
Observation {action_id, status, structured, artifact_refs, diagnostics}
Artifact {uri, hash, media_type, producer, classification}
Checkpoint {run_id, state_version, event_offset, pending_effects}
PolicyDecision {action_id, decision, constraints, policy_version}
VerificationResult {contract, checks, status, evidence_refs}
```

```
while run.active:
    context = compiler.build(run.state, budget)
    proposal = model.decide(context, tool_views)
    if proposal.action:
        action = normalize_and_validate(proposal.action)
        decision = policy.evaluate(action)
        observation = executor.commit(action, decision)
        ledger.append(action, decision, observation)
        state.reduce(observation)
    else:
        candidate = seal(proposal.output, state.artifacts)
        return completion_gate.verify(candidate)
```

```
recover(run_id):
    checkpoint = store.latest(run_id)
    replay_pure_events(checkpoint.offset)
    for effect in checkpoint.pending_effects:
        reconcile_by_idempotency_key(effect)
    resume_with_fresh_policy_and_credentials()
```

这些接口是语义合同，不要求所有 runtime 使用同一编程语言或序列化格式。

附录 B：Harness 架构评审检查表

任务与完成

- 是否有版本化目标、交付物、不变量和验收条件？
- 模型停止是否与业务完成分离？
- 是否从目标环境回读结果并生成 evidence package？

上下文与记忆

- 上下文来源、优先级、token 成本与 provenance 是否可见？
- 压缩后哪些状态仍是权威？
- 长期记忆是否有写入门、TTL、纠错和删除？

工具与环境

- Action schema、错误分类和输出截断是否稳定？
- 工具副作用是否幂等、可对账？
- workspace、文件、网络、进程和资源是否隔离？

权限与安全

- 身份、授权、批准和隔离是否分层？
- 凭证是否短期、窄范围且不进入模型上下文？
- MCP、skill、plugin 是否有版本、签名、权限和撤销？
- 是否测试间接 prompt injection 与跨域数据流？

Durable 与多 Agent

- 是否有 checkpoint、取消、恢复、reconciliation？
- 委派是否缩权、限预算、结构化交付？
- 并行写入是否隔离，合并后是否重验？

Eval 与进化

- Capability、regression、安全集是否分开？
- 是否多 trial、报告成本和关键切片？

- 候选是否无法修改 evaluator、held-out 和 policy root?
- 是否 canary、回滚并保存 lineage?

任何 R3/R4 动作若上述关键问题无答案，不应进入生产自治。

附录 C：术语表

- **Agent**：由模型动态管理工作流、使用工具并改变环境状态的系统。
 - **Harness**：围绕模型的上下文、工具、循环、环境、安全、状态、验证和可观测运行时。
 - **Runtime**：承载 Agent 循环或执行动作的运行组件；本文按语境区分 Agent Runtime 与 Execution Runtime。
 - **ACI**：Agent-Computer Interface，模型与计算环境之间的动作和观察接口。
 - **Completion Contract**：目标、交付物、不变量、验收、证据、权限和停止条件的版本化合同。
 - **Evidence Package**：连接输入、artifact、检查、策略和外部效果的机器可读交付证据。
 - **Effect Ledger**：记录副作用意图、幂等键、提交状态和对账结果的账本。
 - **Compaction**：将长会话转换为可继续工作的较短表示，不等同于长期记忆。
 - **Skill**：可按需加载的程序知识，可能包含指令、脚本和资源。
 - **Handoff**：工作流责任从一个 Agent 转移给另一个 Agent。
 - **Held-out**：候选不可见、用于独立评价的数据或检查。
 - **Canary**：只在受限真实流量部署候选版本并监控。
 - **Harness evolution**：对 prompt、工具、上下文、路由、工作流等运行时组件进行受控优化。
 - **Reward hacking**：提高测量分数但偏离真实目标或破坏评价完整性。
-

研究方法 与 局限

本书采用官方文档、开源仓库、论文和社区材料的分层证据法。产品事实以 2026-08-22 为时间截面；无法验证的内部实现不作为事实。设计原则是作者基于多来源的综合推断。研究资产包括 sources.jsonl、evidence.jsonl 与 claims.jsonl。局限包括产品快速迭代、公开 benchmark 污染、厂商数据选择偏差，以及部分 2026 年进化论文尚缺长期生产复现。

完整参考文献

1. 机构/作者未登记 (n.d.). [DeepSeek Harness official repository](#)
2. 机构/作者未登记 (n.d.). [DeepSeek Harness Architecture](#)
3. 机构/作者未登记 (n.d.). [Cordis Primer](#)
4. 机构/作者未登记 (n.d.). [A Programming Paradigm for Spatiotemporal Composability](#)
5. 机构/作者未登记 (n.d.). [Self-Harness: Harnesses That Improve Themselves](#)
6. 机构/作者未登记 (n.d.). [Self-Evolving Agent Harnesses via Gated Semantic Quality-Diversity](#)
7. 机构/作者未登记 (n.d.). [Living-Harness Is an Interactive-Agent Evolver](#)
8. 机构/作者未登记 (n.d.). [Hierarchical Self-Improvement: A Framework for Task-Specific Evolvable Agent Harnesses](#)
9. 机构/作者未登记 (n.d.). [Unrolling the Codex agent loop](#)
10. 机构/作者未登记 (n.d.). [Unlocking the Codex harness: how we built the App Server](#)
11. 机构/作者未登记 (n.d.). [Introducing the Codex app](#)
12. 机构/作者未登记 (n.d.). [Harness engineering: leveraging Codex in an agent-first world](#)
13. 机构/作者未登记 (n.d.). [OpenAI Codex official repository](#)
14. 机构/作者未登记 (n.d.). [Continually improving our agent harness](#)
15. 机构/作者未登记 (n.d.). [Dynamic context discovery](#)
16. 机构/作者未登记 (n.d.). [What we learned building cloud agents](#)
17. 机构/作者未登记 (n.d.). [Cursor Developer Habits Report Spring 2026](#)
18. 机构/作者未登记 (n.d.). [OpenHands Runtime Architecture](#)
19. 机构/作者未登记 (n.d.). [OpenHands: An Open Platform for AI Software Developers as Generalist Agents](#)
20. 机构/作者未登记 (n.d.). [Gemini CLI official repository](#)
21. 机构/作者未登记 (n.d.). [OpenCode official repository](#)
22. 机构/作者未登记 (n.d.). [Pi coding agent official repository](#)
23. 机构/作者未登记 (n.d.). [ReAct: Synergizing Reasoning and Acting in Language Models](#)
24. 机构/作者未登记 (n.d.). [Toolformer: Language Models Can Teach Themselves to Use Tools](#)
25. 机构/作者未登记 (n.d.). [Reflexion: Language Agents with Verbal Reinforcement Learning](#)
26. 机构/作者未登记 (n.d.). [Voyager: An Open-Ended Embodied Agent with Large Language Models](#)
27. Richard Fikes; Nils Nilsson (1971). [STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving](#)
28. Reid G. Smith (1980). [The Contract Net Protocol](#)
29. 机构/作者未登记 (2020). [BDI Agent Architectures: A Survey](#)
30. 机构/作者未登记 (2024). [SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering](#)
31. 机构/作者未登记 (2026). [Pi coding agent official repository](#)

32. 机构/作者未登记 (n.d.). [GitHub repository metadata for OpenHands](#)
33. 机构/作者未登记 (n.d.). [GitHub repository metadata for Gemini CLI](#)
34. 机构/作者未登记 (n.d.). [GitHub repository metadata for OpenCode](#)
35. 机构/作者未登记 (n.d.). [GitHub repository metadata for Pi](#)
36. 机构/作者未登记 (2023). [Original BabyAGI archived implementation](#)
37. 机构/作者未登记 (2023). [AutoGPT official repository](#)
38. 机构/作者未登记 (2023). [Autonomous Agents and Agent Simulations](#)
39. 机构/作者未登记 (2023). [AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation](#)
40. 机构/作者未登记 (2025). [Building LangGraph: Designing an Agent Runtime from First Principles](#)
41. 机构/作者未登记 (n.d.). [Aider Repository Map](#)
42. 机构/作者未登记 (n.d.). [Aider GPT Code Editing Benchmarks](#)
43. 机构/作者未登记 (n.d.). [Executable Code Actions Elicit Better LLM Agents](#)
44. 机构/作者未登记 (n.d.). [AI Harness Engineering: A Runtime Substrate for Foundation-Model Software Agents](#)
45. 机构/作者未登记 (n.d.). [Building Effective Agents](#)
46. 机构/作者未登记 (n.d.). [How the Claude Agent SDK loop works](#)
47. 机构/作者未登记 (n.d.). [How Claude Code works](#)
48. 机构/作者未登记 (n.d.). [Idempotency and retries in durable execution](#)
49. 机构/作者未登记 (2024). [Lost in the Middle: How Language Models Use Long Contexts](#)
50. 机构/作者未登记 (2023). [MemGPT: Towards LLMs as Operating Systems](#)
51. 机构/作者未登记 (n.d.). [How Claude remembers your project](#)
52. 机构/作者未登记 (n.d.). [Explore the Claude Code context window](#)
53. 机构/作者未登记 (n.d.). [Model Context Protocol Architecture](#)
54. 机构/作者未登记 (n.d.). [Model Context Protocol Schema Reference](#)
55. 机构/作者未登记 (n.d.). [MCP Authorization](#)
56. 机构/作者未登记 (n.d.). [Scale to many tools with tool search](#)
57. 机构/作者未登记 (n.d.). [DeepSeek Harness Tool Runtime and Code Mode](#)
58. 机构/作者未登记 (n.d.). [Beyond permission prompts: Claude Code sandboxing](#)
59. 机构/作者未登记 (n.d.). [Codex ExecPolicy](#)
60. Carlos E. Jimenez et al. (2023). [SWE-bench: Can Language Models Resolve Real-World GitHub Issues?](#)
61. OpenAI (2024). [Introducing SWE-bench Verified](#)
62. OpenAI (2026). [Why SWE-bench Verified no longer measures frontier coding capabilities](#)
63. Anthropic (2026). [Demystifying evals for AI agents](#)
64. Koki Wataoka, Tsubasa Takahashi, Ryokan Ri (2024). [Self-Preference Bias in LLM-as-a-Judge](#)
65. Lin Shi et al. (2024). [Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge](#)
66. Jonathan Gabor, Jayson Lynch, Jonathan Rosenfeld (2025). [EvilGenie: A Reward Hacking Benchmark](#)

67. Bingchen Zhao et al. (2026). [SpecBench: Measuring Reward Hacking in Long-Horizon Coding Agents](#)
68. Anthropic (2025). [How we built our multi-agent research system](#)
69. Qingyun Wu et al. (2024). [AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation](#)
70. Sirui Hong et al. (2023). [MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework](#)
71. OpenAI (2025). [A practical guide to building AI agents](#)
72. OpenAI (2026). [OpenAI Agents SDK Handoffs](#)
73. Anonymous/Research authors (2025). [Why Do Multi-Agent LLM Systems Fail?](#)
74. Haolun Wu, Zhenkun Li, Lingyao Li (2025). [Can LLM Agents Really Debate? A Controlled Study of Multi-Agent Debate in Logical Reasoning](#)
75. Kunlun Zhu et al. (2025). [MultiAgentBench: Evaluating the Collaboration and Competition of LLM agents](#)